Tottie Master Documentation

NKNGO AI Labs

October 27, 2026

Contents

I Documentation Standards 1

1 Agent Documentation Standards 2
- 1.1 Core Documentation Mandate . . . 2
  - 1.1.1 Rule 1: The LaTeX Mandate . . . 2
  - 1.1.2 Rule 2: The Unified Root (`master.tex`) . . . 2
  - 1.1.3 Rule 3: Post-Implementation Documentation Mandate . . . 2
- 1.2 Process Flow Mapping Standards (The Cookbook Format) . . . 2
  - 1.2.1 Structuring the `process_flows/` Directory . . . 3
  - 1.2.2 The Process Cookbook Format . . . 3
- 1.3 Local Orchestration and Observability Standards (`orch-cli`) . . . 3

II Vision 5

2 Why Tottie Exists 6
- 2.1 The Problem: Agents Have Amnesia . . . 6
- 2.2 A Gallery of Failures (and What Fixes Each) . . . 6
- 2.3 The Thesis: Intelligence at Write Time . . . 7
- 2.4 Positioning Against the 2026 Field . . . 7
- 2.5 Why It Matters (the human framing) . . . 7

III The Cognition Model 8

3 The Cognition Model: What to Remember, and How a Cue Recalls 9
- 3.1 The Core Realisation: RAG Is One Leg, Not the Animal . . . 9
- 3.2 The Governing Principle: Simulate the Solutions, Never the Constraints . . . 9
- 3.3 Availability vs Accessibility: Redefining Forgetting . . . 10
- 3.4 Worth Remembering I: The Salience Gate . . . 10
- 3.5 Worth Remembering II: Surprise (Prediction Error) . . . 10
- 3.6 Worth Remembering III: The Content Taxonomy . . . 11
- 3.7 Memory Becomes Judgment: The `principle` Field . . . 11
- 3.8 How a Cue Retrieves: Typing, Spreading, Completing . . . 11
  - 3.8.1 Cue Typing — "What question is `implement feature A`?" . . . 11
  - 3.8.2 Spreading Activation, Priming, and Pattern Completion . . . 12
  - 3.8.3 Three Legs, Not One . . . 12
- 3.9 Retrieval Is Not a Passive Read . . . 12
- 3.10 The Ranking Model ("Forgetting") . . . 12
- 3.11 Consolidation: The Sleep Job . . . 12
- 3.12 Reconsolidation: Updating on Access . . . 13
- 3.13 Prospective Memory: Remembering Intentions . . . 13
- 3.14 Metacognition and Confidence . . . 13

3.15 Transactive Memory: Knowing Where . . . 13
3.16 Working Memory and Attention Gating . . . 13
3.17 The Full Memory Taxonomy, Mapped . . . 13
3.18 The Component Roster . . . 14
3.19 Neuroscience Basis, and Where We Improve On It . . . 14

IV Business and Hackathon 16

4 Hackathon Rules, Dates, and Judging 17
4.1 Problem Statements . . . 17
4.2 Mandatory Meta Tooling . . . 17
4.3 Dates (all times Eastern) . . . 17
4.4 Hard Rules . . . 17
4.5 Sample Anti-Projects (things to avoid resembling) . . . 18
4.5.1 How Tottie differs from the banned “Basic RAG Application” . . . 18
4.6 Judging Rubric (equally weighted, 0–10) . . . 18
4.7 Prizes We Target . . . 18

5 Hackathon Compliance Ledger 19
5.1 Statement of Original Contribution . . . 19
5.2 Feature → Commit Ledger . . . 19
5.3 Reuse Disclosure . . . 19

V System Architecture 20

6 System Architecture 21
6.1 Overview . . . 21
6.2 The Component Roster . . . 21
6.3 Runtime Shape . . . 22
6.4 Module Map (file → component) . . . 22
6.5 The Tier Model . . . 23
6.6 Reuse Map (patterns, not files) . . . 23

7 Data Model and Storage 24
7.1 Storage Decision (documented exception) . . . 24
7.2 Schema . . . 24
7.3 Indexes (and why each exists) . . . 25
7.4 The Field Semantics That Matter . . . 26
7.5 Temporal Correctness . . . 26
7.6 Owner Scoping . . . 26
7.7 Embeddings . . . 26

8 Model Providers and the Intelligence Layer 27
8.1 Provider Abstraction . . . 27
8.2 Component Model Choices . . . 27
8.3 Meta Depth (the rubric column) . . . 27
8.4 Verified Endpoint (2026-10-27) . . . 28
8.5 Cost Control . . . 28
8.6 Optional: MCP Sampling (flagged, off by default) . . . 28

9 MCP Surface **29**
9.1 Tools . . . 29
9.2 Resources . . . 29
9.3 Prompt . . . 29
9.4 Dual-Path Recall (the design answer) . . . 30
9.5 Retrieval Is Cue-Typed, Not Just Similar . . . 30
9.6 Confidence in the Response . . . 30

10 HTTP API: Endpoints, Events, and Errors **31**
10.1 Endpoint Map . . . 31
10.2 Request and Response Shapes . . . 31
10.3 The Event Stream . . . 32
10.4 Errors . . . 32
10.5 Cross-Origin and Health . . . 32

11 Runtime: Where Tottie Lives and What Drives It **33**
11.1 The Question . . . 33
11.2 What the Rules Actually Require . . . 33
11.3 Option A: Local-First (the decision) . . . 33
11.4 Option B: Hosted (rejected for the hackathon) . . . 33
11.5 Option C: The Brain Question — Own Chat, Hijacked Agent, or Both? . . . 33
11.6 The Research Consensus . . . 34
11.7 What Films Where . . . 34
11.8 Post-Hackathon Path . . . 34

12 Security: Provenance, Trust, and Poisoning **35**
12.1 The Threat: Memory Poisoning . . . 35
12.2 Control 1: Provenance and Trust Levels . . . 35
12.3 Control 2: Quarantine . . . 35
12.4 Control 3: Memory Content Is Data, Not Instructions . . . 35
12.5 Authentication and Rate Limits . . . 36
12.6 Privacy and the Right to Be Forgotten . . . 36
12.7 What We Deliberately Do Not Do . . . 36

13 Performance, Cost, and Resilience **37**
13.1 The Non-Blocking Write Path . . . 37
13.2 Budgets . . . 37
13.3 Bounded Recall . . . 37
13.4 Cost Control . . . 38
13.5 Storage Engineering . . . 38
13.6 Resilience . . . 38
13.7 Observability . . . 38

14 Tottie UI and the Live Memory Panel **39**
14.1 Layout . . . 39
14.2 The Panel Is a Live Stream of `memory_events` . . . 39
14.3 The Brain Graph (3D) . . . 40
14.4 The Sleep Button . . . 40
14.5 Tottie Avatar — Drop-In Sprite (frame sets per state) . . . 40
14.6 Ablation Toggle . . . 40

VI Process Flows **41**

15 Process Flow: The Write Path (Non-Blocking) **42**
15.1 The Host Posts a Turn . . . 42
15.2 A Worker Drains the Queue . . . 42
15.3 Trust and Quarantine . . . 42
15.4 Structure, Embed, Reconcile . . . 42
15.5 Failure Fallback . . . 43
15.6 Streaming Back . . . 43

16 Process Flow: Surprise Gating and Ranking ("Forgetting") **44**
16.1 The Attention and Surprise Gate . . . 44
16.2 A Surprising Turn Is Encoded . . . 44
16.3 Ranking, Not Deleting . . . 44
16.4 The Cue Always Works . . . 45
16.5 Interference Control . . . 45

17 Process Flow: The Recall Path (Bounded) **46**
17.1 A New Turn Arrives . . . 46
17.2 The Identity Slice Is Always Included . . . 46
17.3 Three Legs in One Round-Trip . . . 46
17.4 Spreading Activation and Completion . . . 46
17.5 Confidence (Metacognition) . . . 47
17.6 Curate and Inject . . . 47
17.7 Prospective Check (no scheduler) . . . 47
17.8 The Retrieval Story (3-stage chain) . . . 47
17.9 The Deliberate Probe . . . 47

18 Process Flow: Cue Typing, Spreading Activation, and Pattern Completion **48**
18.1 The User Types an Instruction, Not a Question . . . 48
18.2 The Deterministic Rules (implementation contract) . . . 48
18.3 Seeding the Associative Network . . . 49
18.4 Spreading Activation . . . 49
18.5 Pattern Completion . . . 49
18.6 Curate and Inject . . . 49
18.7 Temporal Scope: The "What Did I Just Do" Branch . . . 49
18.8 Inhibition and Abstention . . . 50

19 Process Flow: Reconcile and Temporal Correctness **51**
19.1 The Decision Table . . . 51
19.2 Worked Example: "I'm vegetarian now" . . . 51
19.3 Why This Is Hard and Why It Wins . . . 52

20 Process Flow: Reconsolidation on Access **53**
20.1 A Memory Is Retrieved . . . 53
20.2 The User Contradicts It . . . 53
20.3 Why It Is Different . . . 53

21 Process Flow: Consolidation (the Sleep Job) **54**
21.1 The User Presses "Sleep" . . . 54
21.2 Semanticisation . . . 54
21.3 Ranking and Scaling . . . 54
21.4 Why This Beat Exists . . . 54

22 Process Flow: The Demo Click-Path **55**
22.1 Beat 1 — Cold Open (Amnesia) . . . 55
22.2 Beat 2 — Tottie Wakes (Identity Pinning) . . . 55
22.3 Beat 3 — Write-Time Magic (Supersede) . . . 55
22.4 Beat 4 — Three Bodies (Cross-Agent Recall) . . . 55
22.5 Beat 5 — Autosave a Mistake . . . 55
22.6 Beat 6 — Restart + Ablation . . . 56
22.7 Beat 7 — The Joke and the Pitch . . . 56

VII Planning **57**

23 Roadmap **58**
23.1 Scope Tiers (9 days, solo, alongside work and study) . . . 58
23.2 Revised Technical Build Order . . . 58
23.3 Phase Schedule . . . 58
23.4 Gating . . . 59
23.5 Eval Harness Contract . . . 59
23.6 Mandates Honoured During the Build . . . 59

24 Execution Plan: Waves, Gates, and Proof **60**
24.1 Objective and Constraints . . . 60
24.2 The Waves . . . 60
24.3 Wave-to-Phase Map . . . 61
24.4 Revision Gates . . . 61
24.5 Coverage and Estimation Discipline . . . 61
24.6 Test Strategy . . . 61
24.7 Validation Commands . . . 62
24.8 Risks and Relief Valves . . . 62

25 Master Todo **63**
25.1 Phase 0 — Scaffold . . . 63
25.2 Phase 1 — Documentation . . . 63
25.3 Phase 2 — Deck . . . 63
25.4 Phase 3 — Core Engine (the gate + the minds) . . . 63
25.5 Phase 4 — MCP . . . 63
25.6 Phase 5 — Round-Trip . . . 63
25.7 Phase 6 — UI . . . 64
25.8 Phase 7 — Reconcile / Tags / Identity / Links / Rank . . . 64
25.9 Phase 8 — Cognition + Integration + Eval . . . 64
25.10 Phase 9 — Film . . . 64
25.11 Phase 10 — Submit . . . 64

26 Demo Script (3 minutes, screen capture + captions + music) **65**
26.1 Shot List . . . 65
26.2 Narration Captions (verbatim) . . . 66
26.3 Music and Pacing . . . 66
26.4 Filming Protocol (known vs unknown scenarios) . . . 66

27 Plan Audit (Numbered Outline and Method) **67**
27.1 Audit Method . . . 67
27.2 Independent-Review Questions (for third-party judges) . . . 67
27.3 The Numbered Outline . . . 67

27.4 Audit Ledger 68
27.5 v6 Audit Ledger (pre-build, 2026-10-26) 68
27.6 v7 Research Ledger (3D brain graph + retrieval story, 2026-10-27) 69
27.7 v8 Planning Ledger (Joel + Shape Up + WBS + risk audits, 2026-10-27) 69
27.8 v9 Final Ledger (pre-development consistency sweep, 2026-10-27) 70
27.9 v10 Incident Ledger (the YOLO false-block, 2026-10-27) 70
27.10v11 Build Ledger (Meta live + keys cascade + W0 green, 2026-10-27) 71

Part I

Documentation Standards

Chapter 1

Agent Documentation Standards

1.1 Core Documentation Mandate

This document serves as the absolute source of truth for any Autonomous AI Agent (including Gemini, Loid3, or other models) operating across the ENTIRE NKNGO Portals Ecosystem (including NKNGO-LANG, LearnersQuizNamibia, Pattern, and all future projects). When asked to write, update, or analyze documentation, you MUST adhere to the following universal standardization rules without exception.

1.1.1 Rule 1: The LaTeX Mandate

- **No Markdown Allowed:** All architectural, system, and process flow documentation MUST be written in LaTeX(`.tex`) format.
- **Exceptions:** The **only** Markdown files permitted in the root directory are `README.md` (for GitHub previews) and `GEMINI.md` (the immutable core system prompt for agents).
- **Deletion Clause:** If an agent generates or discovers standalone `.md` files containing system documentation, they must convert them to `.tex`, move them into the `docs/` directory, and permanently delete the `.md` file.

1.1.2 Rule 2: The Unified Root (`master.tex`)

- All documentation lives inside the `/docs` directory.
- You must never create isolated `.tex` files. Every new file must be logically categorized into a subdirectory (e.g., `process_flows/`, `business/`, `system/`) and formally imported via an `\input{}` command inside `docs/master.tex`.

1.1.3 Rule 3: Post-Implementation Documentation Mandate

- Whenever an agent implements a new feature, refactors a system, or writes new code, they MUST immediately document the flows and architecture in the appropriate LaTeX files. Do not wait for the user to ask for documentation after coding; it is an automatic, mandatory requirement.

1.2 Process Flow Mapping Standards (The Cookbook Format)

When mapping flows, DO NOT generate massive dumps of raw code, line-by-line syntax, or meaningless high-level abstractions. Documentation must represent the **Human/System Process Journey**. We are writing cookbooks that map how a user gets from A to B so we can understand the architecture before coding.

1.2.1 Structuring the `process_flows/` Directory

Flows must be organized by logical **Domains**, not arbitrarily by file name. Example structure:
- `docs/process_flows/tasks_and_projects.tex`
- `docs/process_flows/finance_and_ledgers.tex`
- `docs/process_flows/social_and_friends.tex`
- `docs/process_flows/system_and_sync.tex`

1.2.2 The Process Cookbook Format

Every feature within a domain must be broken down into explicit, human-readable steps that explain the complete logic loop. Use standard LaTeXlists and sections.

CRITICAL ANTI-ABSTRACTION MANDATE: Do **NOT** write code-focused abstracts for the UI/Action layers (e.g., *"The getChildren() method of the NoteProvider fires"*). You **MUST** describe the physical human action (e.g., *"The user expands the Notes folder in the sidebar"*).

BRUTAL UI GRANULARITY MANDATE: Never map flows at a surface level (e.g., "The user creates an account" or "The user logs an expense"). You **MUST** map the exact, granular physical UI elements the user interacts with. This means explicitly documenting the specific horizontal scrolling chip selectors, the exact preset labels (e.g., FNB, Bank Windhoek), the exact toggles for recurring actions (e.g., Weekly vs Monthly), and the specific text inputs (e.g., "Quantity" and "Cost Basis"). If the UI has a specific switch or dropdown, the documentation must explicitly state it. Keep mapping and "adding on until it's really all there".

Required elements for EVERY mapped feature:
1. **Trigger:** How does the process start physically? (e.g., "User swipes left on a Todo in the Inbox")
2. **Action / UI State:** What does the user see and interact with? (e.g., "Action bubble menu appears, user selects 'Bump to Tomorrow'")
3. **System Logic:** What is the intended behavior and calculation behind the scenes? (e.g., "System recalculates the due date to tomorrow at 00:00:00 and removes the item from the Today deck")
4. **Data Flow & Mutators:** Where does the data go? (e.g., "Updates local WatermelonDB SQLite -> Queues offline mutation in AsyncStorage -> Synchronizes via POST /api/sync to Drizzle ORM when online")

CRITICAL CROSS-SCREEN MANDATE: Flows are highly dynamic and rarely siloed to a single screen. You **MUST** explicitly map deep inter-screen connections (e.g., "Notes flow pushes parameters to FinanceScreen to auto-add expenses" or "Project Focus forces the global Calendar to filter by Project ID"). Never stop at the boundary of a single screen if the user journey naturally bridges into another domain.

Failure to adhere to the Domain Structure, the Brutal UI Granularity Mandate, and the Cookbook Format will result in bloated, unreadable documentation and constitutes a complete failure of the agent's architectural mandate.

1.3 Local Orchestration and Observability Standards (`orch-cli`)

All projects across the NKNGO Portals ecosystem are orchestrated locally via `orch-cli` located in `launcher/orch-cli`. Autonomous AI Agents MUST NEVER guess whether a server, bundler, or service is functioning. Agents must operate with absolute physical observability:
1. **Status Inspection:** Run `orch status` (or `orch status --json`) to verify the active daemon, port, PID, profile, and all registered child processes.
2. **Error Isolation:** Run `orch logs -e` to isolate errors, unhandled exceptions, and 500s across all frameworks without wading through routine log clutter.
3. **Browser Telemetry:** Run `orch logs BROWSER -n 50` to inspect live client-side Chrome console errors, React component crashes, and failed network requests.

4. **Service-Specific Logs:** Run `orch logs <SERVICE> -n 50` (e.g., `orch logs MOBILE-WEB` or `orch logs API`) to audit backend and bundler output.
5. **Hot Reloading:** Run `orch restart` to signal the active daemon to reload all child services upon applying code fixes.
6. **Profile Switching:** Launch targeted multi-service environments with `orch run -p "<profile>"` (e.g., `orch run -p "Pattern + Ask AI (Test Local)"`).

Part II

Vision

Chapter 2

Why Tottie Exists

2.1 The Problem: Agents Have Amnesia

Every agent on every platform wakes up stateless. It has no history, no user context, and no memory of yesterday. A year of conversations later, it still does not know who you are, what you build, or what you decided. All of that intelligence, yet it fails the baseline requirement of an actual mind: **continuity**.

The naive fix is a bigger context window. That is not memory — it is an expensive scratchpad. As input length grows, cost rises linearly, latency rises super-linearly, and accuracy *drops* (the "lost in the middle" effect). Long context is a tax with diminishing returns.

2.2 A Gallery of Failures (and What Fixes Each)

Every row below is a failure the Architect has personally watched an agent commit — most recently, an agent that could not say what it had done a few runs earlier *in the same conversation*. That one sent us back to the neuroscience (NotebookLM "META hackathon"): the diagnosis is a **retrieval failure, not an encoding failure**. The trace is *available* (it sits in the session log, and the agent recognises it when shown), but *inaccessible* to unprompted recall — the generate-recognise model fails at Stage 1 (search/generation), while recognition bypasses that stage entirely. Humans solve the same problem with **serial recall** over episodic traces; we transfer exactly that, minus the 15–30 second buffer and the transposition errors.

Symptom (user-visible)	Human analogue	Tottie fix
"What did I just do?" — forgot work from a few runs ago	Serial recall + recency	Session recap: ordered replay of the session's event chain
"What is my name again?" — re-asks settled facts	Identity / autobiographical memory	Identity tier, pinned raw, zero search
Says X on Monday, not-X on Tuesday	Reconsolidation on contradiction	Reconciler: SUPERSEDE + validity window, old kept
Fixes the same bug every week	Procedural memory (skills)	Lessons tier + autosave: error $\rightarrow$ fix
Answers confidently when it knows nothing	Metacognition (feeling of knowing)	Confidence from the margin; abstain below 0.35
Buries the answer in 200k tokens (lost in the middle)	Attention gating	Write-time curation; 400-token card
Repeats a lie it once read	Source monitoring	Provenance + quarantine; low-trust never injected
Does something 12 times, learns nothing	Sleep consolidation	Consolidator: episodes $\rightarrow$ semantics

"Remind me Friday" — Friday comes, silence	Prospective memory	Prospective Queue: cues fire
The chat learns it; the IDE never knows	The shared notebook	One MCP server every host mounts
Insisted it was blocked for 3 minutes while unlocked the whole time	False self-model + source-monitoring failure (trusted the error text, ignored the status bar)	Procedural lesson autosaved on correction; retrieved on the next matching error

2.3 The Thesis: Intelligence at Write Time

Standard retrieval-augmented generation (RAG) applies intelligence at **read time**: the agent searches when it needs something. Tottie flips that. It applies intelligence at **write time**: memory curates itself as you live, deciding what matters *before* it is needed. The agent does not manage memory. It just remembers.

- **Observer** runs in parallel and decides what is worth keeping.
- **Scribe** structures, tags, embeds, and *reconciles* what was kept.
- **Retriever** runs on every turn and surfaces what is contextually relevant.
- **Curator** writes the compact *Context Card* the agent actually receives.

2.4 Positioning Against the 2026 Field

The field has converged on a name for this pattern: **learned consolidation**. Live capture $\rightarrow$ LLM extraction of facts and decisions $\rightarrow$ reconciliation (update vs supersede) $\rightarrow$ episodic/semantic split $\rightarrow$ dual retrieval. mem0, Zep/Graphiti, and A-Mem all instantiate it. Published figures show why it matters: mem0 reports **91% lower p95 latency** and **90% fewer tokens** than full-context, for roughly a six-point J-score trade on LOCOMO.

Tottie's two differentiators:

1. **Write-time salience + reconciliation is the product**, not a bolt-on. The system is judged on the decisions it makes at ingestion.
2. **It is transport-agnostic (MCP).** The same brain mounts into a chat client, an IDE, or any MCP host. Memory is infrastructure, not an app feature.

2.5 Why It Matters (the human framing)

The best context is the context you can *automatically* give an agent, and that context is almost always about the person using it. Tottie is one store that knows your life, your preferences, your decisions, and your infrastructure — so a chat assistant remembers your partner's birthday and a coding agent already knows your stack and database choice before it writes a line.

Part III

The Cognition Model

Chapter 3

The Cognition Model: What to Remember, and How a Cue Recalls

This chapter is the intellectual core of Tottie. It answers two questions a plain RAG pipeline never asks: *what is worth remembering*, and *how the right memory comes back*. It also records which mechanisms we **copy** from the brain, which we **refuse** to copy, and where we **improve on** the brain because we are not bounded the way it is.

3.1 The Core Realisation: RAG Is One Leg, Not the Animal

Retrieval-augmented generation matches **text similarity**. That fails the moment the input is not a question. "Implement feature A" has no textually similar neighbour, yet a competent human immediately surfaces the project's conventions, past decisions, and pitfalls. Tottie's retrieval is therefore a **staged, associative process**, not one similarity search.

3.2 The Governing Principle: Simulate the Solutions, Never the Constraints

This is the single most important design rule in the project.

> The brain forgets because it is **bounded** — finite neurons, metabolic cost, lossy reconstructive storage. Those are **constraints, not features**. We are not bounded. Storage is nearly free, search is exhaustive, and we have full control. Blindly copying Ebbinghaus decay would be copying a limitation to solve a solution problem. **Simulate the brain's solutions; never simulate its constraints.**

Biological **constraint** (do NOT copy)	Functional **solution** (DO copy)
Bounded capacity and metabolic cost	Spreading activation over an associative graph
Passive trace decay	Complementary fast/slow consolidation
Lossy reconstruction and confabulation	Autoassociative pattern completion
Interference and blending	Surprise / prediction-error gating
Tip-of-the-tongue blocking	Reconsolidation and dynamic patching
Human working-memory span (7 ± 2)	Salience and importance tagging

Concretely, Tottie **refuses** to copy: uncontrolled time-based decay of raw records; reconstructive guessing inside the store; retrieval blocking; and an artificially small working set. It **does** copy: spreading activation, consolidation, pattern completion, surprise gating, reconsolidation, and salience.

3.3 Availability vs Accessibility: Redefining Forgetting

Tulving and Pearlstone separated two states of stored information:

- **Availability** — the trace was encoded and still exists.
- **Accessibility** — the trace can be retrieved *right now*, which depends on whether the current cues match the encoding.

Retrieval failure (cue-dependent forgetting) is an available-but-inaccessible memory. The **tip-of-the-tongue** state is its poster child: you know the name is in there, you can feel its shape, and you still cannot produce it.

For a machine, **forgetting should not mean deletion at all**. It should mean **control of accessibility**. Everything stays *available* (stored, recoverable with the right cue); only the cued subset is *accessible* at any moment. The tip-of-the-tongue feeling is a *bug*; we keep the mechanism and drop the bug — the cue always works.

Traditional deletion (human / legacy stores)	Accessibility control (Tottie)
Hard physical erasure	Lower the memory's retrieval ranking
Irreversible loss of raw data	Filter it out of the attention window
"Prevents clutter" by destroying evidence	Compress it into an abstract schema

So "forgetting" in Tottie is three operations, none of which is a shredder:

1. **Ranking / salience modulation** — a utility factor lowers a stale memory's search weight, so it drops below the surface threshold but stays in the store.
2. **Attention gating** — the item is kept out of the active context window.
3. **Consolidation / compression** — a detailed episode is summarised into a durable semantic node, with the raw episode retained behind it.

The **Decayer** is therefore really a **Ranker** + **Compressor**, not a shredder. Hard deletion happens in exactly one case: an explicit right-to-be-forgotten request.

3.4 Worth Remembering I: The Salience Gate

The brain retains an experience when activity is strong *and* neuromodulators (amygdala, norepinephrine, dopamine) lower the threshold for durable long-term potentiation; weak activity produces long-term depression and is pruned. The Observer is that gate:

```
salience = surprise * (w1*persistence + w2*reusability + w3*consequence +
↪  w4*arousal)
```

Below threshold → DISCARD. The weights are tuned by the evaluation harness.

3.5 Worth Remembering II: Surprise (Prediction Error)

Under **predictive coding** the brain encodes only the *prediction error*; expected input is suppressed (repetition suppression). Tottie computes a **surprise score** before storing:

```
prediction_error = 1 - cos(turn_embedding, predicted_embedding)
encode only if prediction_error >= SURPRISE_FLOOR
```

Routine turns are suppressed; surprising ones are encoded. **Reward prediction error** tags corrections, feedback, and surprising successes or failures for immediate consolidation. Surprise is the *multiplicative* term in the salience equation — the most important mechanism the first draft lacked.

3.6 Worth Remembering III: The Content Taxonomy

Kind	Example	Default tier
Preference (style/tone/format)	"Oxford comma; no em-dashes; open with a TL;DR."	identity
Convention / rule	"Always run tests before commit."	identity
Binding / fact	"Project A uses Postgres; Project B uses Mongo."	knowledge
Decision (+ principle + rationale)	"Chose pgvector for C: already running locally."	knowledge
Constraint	"The service container is 1 CPU / 1 GB."	knowledge
Identity / persona	name, timezone, "her birthday is Nov 7".	identity
Episodic event	"On Oct 29 we decided X in session Y."	episodic
Lesson / procedure	`mcp.server.fastmcp` import error → pin `mcp<2`.	procedural
Intention (prospective)	"Remind me to ask about the deploy on Friday."	prospective

Per-persona emphasis: a *writer* needs preferences, conventions, and style exemplars; a *developer* needs bindings, conventions, decisions-with-principle, and lessons; a *person* needs identity, preferences, dates, and relationships.

3.7 Memory Becomes Judgment: The `principle` Field

"For project C, use project A's database because it is better" is **not recall** — it is projecting a stored principle onto a new situation. Decisions are stored as `decision + principle + rationale`, and the Curator surfaces **principles** whenever the cue is a decision.

3.8 How a Cue Retrieves: Typing, Spreading, Completing

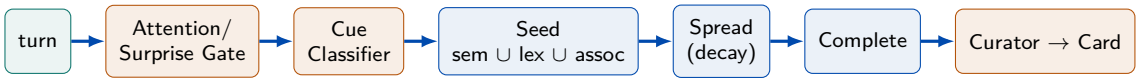


Figure 3.1: Retrieval is staged: gate, classify, seed, spread, complete, curate.

3.8.1 Cue Typing — "What question is `implement feature A`?"

It is not a question. The turn is classified; the type selects the retrieval policy.

Cue type	Example	Retrieval policy
QUESTION	"When is her birthday?"	entity facts + identity + knowledge
COMMAND / INTENT	**"Implement feature A."**	**project bindings + conventions + past decisions on A + lessons + constraints**

STATEMENT (new info)	"I moved to Windhoek."	write path; minimal recall
CORRECTION	"Actually, it is the 8th."	recall the contradicted memory → supersede
REFERENCE	"Use that DB we discussed."	resolve the referent from recent episodic memory
CHITCHAT	"thanks!"	none (do not pollute)

3.8.2 Spreading Activation, Priming, and Pattern Completion

From the seeded nodes, activation propagates along the edges of `memory_links`, decaying with each hop, and priming lowers the threshold for linked concepts — why a memory from fifteen years ago surfaces the instant someone mentions a band. The **Completer** then reconstructs the whole coherent cluster (**pattern completion**) rather than three loose chunks.

3.8.3 Three Legs, Not One

Tottie runs **semantic** (embeddings), **lexical** (full-text), and **associative** (graph activation) retrieval in parallel and fuses them. RAG is retained — it is simply no longer alone. If nothing clears the activation floor, Tottie **abstains** rather than injecting noise.

3.9 Retrieval Is Not a Passive Read

- **Testing effect:** a successful recall *increments utility* — remembering makes a memory more likely to be remembered again.
- **Retrieval-induced forgetting:** a recall *down-weights the near-duplicate competitors* it passed over, sharpening subsequent searches.
- **Spacing effect:** Tottie schedules **spaced reviews** at expanding intervals rather than trusting one write.

3.10 The Ranking Model ("Forgetting")

A memory's **utility** — a *ranking* signal, never a delete signal — decays with age and rises with use:

```
utility = exp(−ln2 ∗ age / half_life) ∗ (1 + log(1 + access_count))
archive (deprioritise) when utility < MIN_UTILITY   # NOT delete
```

Low-utility memories remain **available** and simply fall out of the accessible set until a cue revives them. **Interference control** (sparse pattern separation on write, category filtering on read) keeps overlapping memories from blending, and **retrieval-induced forgetting** down-weights competitors by using them.

3.11 Consolidation: The Sleep Job

The brain replays the day during sleep (hippocampal sharp-wave ripples driving cortical replay), semanticising episodic traces into general knowledge. Tottie's **Consolidator** is an on-demand "Sleep" job that (1) replays recent episodic memories, (2) extracts recurring structure into **semantic** memories, (3) schedules **spaced reviews**, and (4) compresses low-utility detail — *retaining* the raw episodes behind the summary. It can also **self-test**.

3.12 Reconsolidation: Updating on Access

When a memory is retrieved it is marked labile for the remainder of the turn. If the user then contradicts it, the active trace is **patched** (superseded) rather than merely appended, and adjacent nodes are co-activated. Supersession is *retrieval-triggered*, not only write-triggered. **Unlike the brain, nothing is lost:** the superseded version is kept and versioned — we non-destructively reconsolidate.

3.13 Prospective Memory: Remembering Intentions

Tottie adds a **Prospective Queue**: intentions carry an **event cue** or a **time cue** and fire *proactively* when the cue occurs — a genuine product feature ("remind me to…").

3.14 Metacognition and Confidence

A **Metacognitive Evaluator** computes **confidence** from the retrieval margin and selects an action: *HIGH* $\rightarrow$ generate; *LOW* $\rightarrow$ verify, ask, or abstain. This is the difference between an agent that hallucinates and one that says "I don't have that."

3.15 Transactive Memory: Knowing Where

Wegner's *transactive memory* is memory for *who knows what*. The design calls for a **Transactive Directory** mapping capabilities to where the knowledge lives (a tool, a document, an external system), so it routes questions instead of hoarding everything locally. It is **cut for the hackathon** (documented, not built); the shared MCP store covers the demo need.

3.16 Working Memory and Attention Gating

Baddeley's model separates a limited **working memory** from long-term storage. Tottie keeps the *solution* — an **attention gate** and a binding step — but **refuses the span limit**: we do not constrain the active set to 7 ± 2 items.

3.17 The Full Memory Taxonomy, Mapped

Human system	Description	Tottie
Episodic (declarative)	dated personal events	**episodic** tier
Semantic (declarative)	general facts, meanings	**knowledge** tier
Autobiographical	episodic + personal semantic	identity + episodic
Procedural (non-decl.)	skills, habits, how-to	**procedural** (Lessons) tier
Priming (non-decl.)	lowered thresholds after exposure	**Activator** priming
Conditioning (non-decl.)	stimulus–response associations	`memory_links`
Working memory	bounded active workspace	the present thought + attention gate *(unbounded)*
Prospective	future intentions	**Prospective Queue**
Transactive	directory of who knows what	**Transactive Directory**

3.18 The Component Roster

Component	Job	Tier
Attention Gate	filter incoming turns before storage	MUST
Surprise Gate	prediction-error score; encode only the unexpected	MUST
Cue Classifier	intent type + entities → retrieval policy	MUST
Observer	is this worth remembering?	MUST
Salience Scorer	the rubric, with surprise as a multiplier	MUST
Scribe	structure, tag, embed, route, sparse-separate	MUST
Reconciler	reconsolidation: patch the labile trace (non-destructive)	MUST
Retriever	seed + hybrid + graph activation	MUST
Activator	spreading activation, priming, decay	MUST
Curator	assemble the Context Card + "why surfaced"	MUST
Completer	pattern completion (cluster reconstruction)	SHOULD
Ranker + Compressor ("Decayer")	deprioritise + compress — **never** shred	SHOULD
Metacognitive Evaluator	confidence from the retrieval margin; gate assertions	SHOULD
Prospective Queue	event/time-triggered intentions	SHOULD
Consolidator	sleep: replay → semanticise → space → compress	SHOULD
Transactive Directory	route to where knowledge lives	CUT (hackathon)

3.19 Neuroscience Basis, and Where We Improve On It

Mechanism	Tottie
Encoding specificity (Tulving)	rich context tags stored at write time
Spreading activation + decay (Collins–Loftus)	the **Activator**
Pattern completion, CA3 autoassociative	the **Completer**
Pattern separation, dentate gyrus	sparse-separate on write
Predictive coding + reward prediction error	the **Surprise Gate**
Complementary Learning Systems + replay	the **Consolidator**
Reconsolidation (Nader)	the **Reconciler**, made non-destructive
Salience / neuromodulation (amygdala, NE)	the **Salience Scorer**
Ebbinghaus curve, interference	the **Ranker** (availability vs accessibility)
Testing effect, spacing, retrieval-induced forgetting	utility boost + spaced review + competitor suppression
Prospective memory	the **Prospective Queue**
Metacognition / feeling of knowing	the **Metacognitive Evaluator**
Transactive memory (Wegner)	the **Transactive Directory**
Working memory (Baddeley)	attention gate, *without the span limit*
Human constraint we improve on	**Tottie advantage**
Bounded capacity	unbounded storage
Trace decay / permanent loss	deprioritise, never destroy

Lossy reconstruction	exact, verbatim retention
Tip-of-the-tongue blocking	the cue always works
Forgetting is involuntary	forgetting is a tunable knob we own

Sources: the NotebookLM notebook "META hackathon" (`248ae575-4f2d-4ef1-b75f-d6ce38bb514a`), now over 35 sources covering hippocampal encoding and engram theory, LTP and plasticity, spreading activation, encoding specificity, consolidation, episodic/semantic/autoassociative memory, reconsolidation, forgetting and the forgetting curve, spaced repetition, the testing effect, interference, predictive coding, working memory, prospective and transactive memory, chunking, metacognition, reconstructive memory, retrieval failure, and availability versus accessibility.

Part IV

Business and Hackathon

Chapter 4

Hackathon Rules, Dates, and Judging

This chapter records the constraints we are optimising against. It is a working extract of the **Meta Global AI Developer Hackathon Participant Guide** so the team never loses sight of the rules.

4.1 Problem Statements

Every project must build toward one of two statements:
- **Build for Your World** — bring Meta's models into a domain you know firsthand.
- **Push the Frontier** — use Meta's models to give a machine a new capability.

Tottie claims *Build for Your World*: an agent memory layer solves a problem the builder knows firsthand (agents that forget), and it generalises to everyone.

4.2 Mandatory Meta Tooling

Projects must incorporate Meta's models: the *Muse Spark* series, *Muse Wonder*, *Muse Image*, *Muse Voice Transcribe*, and *SAM 3.1* via the Meta Model API; run Meta open-weight models (*Muse Glimmer*, *Muse Spark Open*); or build faster with *Muse Code*. Tottie routes its agents (Observer/Scribe/brain) through the Meta Model API, with a local fallback so the demo never dies.

4.3 Dates (all times Eastern)

When	What
Mon Oct 26, 12:00 PM	Kickoff
Tue Oct 27, 12:00–1:00 PM	Agents 101: Build a Multimodal AI Agent
Wed Oct 28, 12:00–1:00 PM	Agents 102: Best Practices for Verifiable Agents
Fri Oct 30, 12:00–1:00 PM	Specialised Intelligence on Fireworks (Muse Open)
Tue Nov 3	Last full day for submission questions
Wed Nov 4, 9:00 PM	**Final office hours; submissions due**
Thu Nov 5 – Sun Nov 8	Judging period
Mon Nov 9, 12:00 PM	Closing ceremony; top 10 announced; winners revealed
Mon Nov 16	Community vote winner announced

Building period: **Oct 27 – Nov 4**. Tottie's initial commit is dated **2026-10-27**.

4.4 Hard Rules

- **Open source:** repositories must be public.

- **Team size:** max four; solo allowed.
- **Demo requirements:** the demo must only highlight features, code, and functionality *built during the Hackathon*. Judges must be able to identify the original contributions. Failure to do so can result in immediate disqualification.
- **New work only:** what is presented must have been built during the event. Existing codebases may be built on, but new work must be clearly identified.
- **Banned:** projects that violate policy, or that use code/data/assets you lack the rights to.

4.5 Sample Anti-Projects (things to avoid resembling)

Basic RAG applications; simple image analysers; generic "AI for education" chatbots; AI job screeners; nutrition coaches; personality analysers; sports analysers; anything political or involving war; medical advice.

4.5.1 How Tottie differs from the banned "Basic RAG Application"

Tottie is *not* a read-time RAG app. It **flips RAG to write time**: the value is the ingestion-time curation decision, and the deliverable is an **MCP server** (infrastructure), not a document-chat UI. This distinction is stated explicitly on Slide 2 of the demo deck.

4.6 Judging Rubric (equally weighted, 0–10)

Category	What it asks
Impact	Real-world potential; who benefits; could people use it; does it fit a statement.
Use of Meta AI Tooling	How creatively Meta's tooling is used — beyond a basic integration.
Technical Depth & Execution	Is the engineering sound and thoughtfully refined; real craft.
Demo	Is it working, impressive, live-safe, and genuinely cool to watch.

4.7 Prizes We Target

Indie Developer track ($400k pool: $150k / $100k / $75k / $50k / $25k), plus bonus awards: **Best Agentic Use-Case** ($5k), **Best Build on Meta Open-Weight Models** ($10k), **Most Viral Project** ($20k), **Community Vote** ($10k).

Chapter 5

Hackathon Compliance Ledger

5.1 Statement of Original Contribution

Every file, commit, and line of code in this repository was authored between **2026-10-27** and **2026-11-04** — the Hackathon building window. No product code pre-existed the event. Where this project reuses architecture, it reuses *patterns* from the author's own repositories (see the Reuse Map), never third-party code or assets the author does not own. This repository is public, satisfying the open-source rule.

5.2 Feature → Commit Ledger

This table is updated at every phase gate. Judges can verify each row against `git log`.

Feature / Component	Commit	Date
Repo scaffold, env, orch config, docs skeleton	`chore: scaffold Tottie repo`	2026-10-27
LaTeX documentation set (`docs/master.pdf`)	`docs: architecture, flows, roadmap, todo`	2026-10-27
Beamer demo deck (`presentation.pdf`)	`docs: demo deck v1`	2026-10-27
Memory engine skeleton (schema, providers, minds)	`feat(core): memory engine skeleton`	2026-10-28
MCP server surface	`feat(mcp): server surface`	2026-10-29
End-to-end memory round-trip	`feat(core): memory round-trip`	2026-10-30
Tottie UI + live Memory Panel + ablation	`feat(ui): Tottie chat + memory panel`	2026-10-31
Reconcile, tag rooms, identity pinning, Lessons	`feat(core): reconcile + tags + lessons`	2026-11-01
MCP-in-IDE integration + eval harness	`feat: mcp-in-agent integration`	2026-11-02
Demo video, README, compliance ledger	`docs: demo video + compliance`	2026-11-03

5.3 Reuse Disclosure

Tottie reuses *architectural patterns* from the author's own repositories. It copies no files. See the Reuse Map in the System Architecture part.

Part V

System Architecture

Chapter 6

System Architecture

6.1 Overview

Tottie is a memory layer any agent can mount. The **core engine** owns the memory lifecycle; the **MCP server** is the transport; the **Tottie UI** and any **IDE coding agent** are consumers that share one store. The engine's behaviour is specified in *The Cognition Model*; its security in *Security*; its speed in *Performance*.

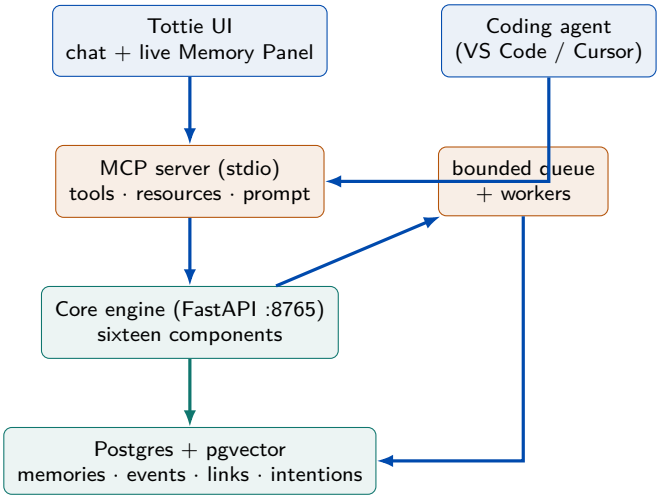


Figure 6.1: One brain, three bodies. Writes are acknowledged immediately and curated asynchronously.

6.2 The Component Roster

Component	Job	Tier
Attention Gate	filter incoming turns before storage	MUST
Surprise Gate	prediction-error score; encode only the unexpected	MUST
Cue Classifier	intent type + entities → retrieval policy	MUST
Observer	is this worth remembering?	MUST
Salience Scorer	the rubric, with surprise as a multiplier	MUST
Scribe	structure, tag, embed, route, sparse-separate	MUST
Reconciler	reconsolidation: patch the labile trace (non-destructive)	MUST
Retriever	seed + hybrid + graph activation	MUST
Activator	spreading activation, priming, decay	MUST

Curator	assemble the Context Card + "why surfaced"	MUST
Completer	pattern completion (cluster reconstruction)	SHOULD
Ranker + Compressor	deprioritise + compress — **never** shred	SHOULD
Metacognitive Evaluator	confidence from the retrieval margin; gate assertions	SHOULD
Prospective Queue	event/time-triggered intentions	SHOULD
Consolidator	sleep: replay → semanticise → space → compress	SHOULD
Transactive Directory	route to where knowledge lives	CUT (hackathon)

6.3 Runtime Shape

The engine is **async throughout**. A single psycopg `AsyncConnectionPool` is created at startup; the embedder is loaded once and called from a thread pool. The write path is a **bounded queue** drained by a small worker pool, so an Observer model call never blocks a user turn. Recall runs inline under a hard wall-clock budget. See *Performance* for the budgets and *Security* for the trust controls that guard what may be written and what may be injected.

6.4 Module Map (file → component)

File	Owns
`core/app/main.py`	FastAPI app, lifespan (pool, embedder, workers), CORS, auth
`core/app/config.py`	typed settings from env (see `.env.example`)
`core/app/db.py`	pool, migrations ledger, query helpers (bound params only)
`core/app/embeddings.py`	FastEmbed bge-small, loaded once, fail-loud, thread-pooled
`core/app/queue.py`	bounded write queue + worker pool + drop-oldest metric
`core/app/agents/gates.py`	Attention Gate, Surprise Gate, heuristic pre-filter
`core/app/agents/observer.py`	salience scoring + candidate proposal (one LLM call, shared with the Scribe)
`core/app/agents/scribe.py`	structure, tag, embed, reconcile (NEW/UPDATE/SUPERSEDE/SKIP)
`core/app/agents/cue_classifier.py`	deterministic cue rules (rules-only for the hackathon)
`core/app/agents/retriever.py`	RRF SQL + graph BFS + confidence
`core/app/agents/curator.py`	rules-based Context Card assembly ($\leq$ 400 tokens)
`core/app/agents/ranker.py`	utility recompute (SQL-side); compression happens in Sleep
`core/app/agents/consolidator.py`	on-demand Sleep job
`core/app/agents/prospective.py`	intention matching on recall/write (no scheduler)
`core/app/routes/*.py`	HTTP surface (see *HTTP API*) + SSE with resume
`core/app/providers/*.py`	`MetaProvider`, `LocalProvider`, `resolve_provider()`
`mcp/server.py`	thin MCP adapter over the same core (9 tools, 2 resources, 1 prompt)
`eval/harness.py`	ON/OFF runner, metrics, special checks (see *Roadmap*)
`core/scripts/seed_demo.py`	offline demo seeding (never imported by runtime code)

6.5 The Tier Model

Identity (persona, pinned, zero search) · **Episodic** (decisions, state, history) · **Knowledge** (documents, semantic facts) · **Procedural** (Lessons / autosave) · **Prospective** (intentions).

6.6 Reuse Map (patterns, not files)

Pattern reused	Source (author's own repo)
RRF hybrid-search SQL shape (two capped CTEs, $k{=}60$)	`ask.nkngo/processor/routers/retrieval.py`
Embedding choice (FastEmbed bge-small)	`ask.nkngo/processor/utils/embeddings.py`
Incremental write + SHA-256 deltas	`ask.nkngo/processor/routers/sync.py`
Relevance floors / abstention	`ask.nkngo/processor/utils/grounding.py`
MCP stdio server scaffold	`Pattern/pattern-vscode-extension/src/mcp-server`

Deliberately not reused: a hardcoded model-cache path, synchronous DB calls inside async handlers, a connection per request, string-interpolated full-text queries, and any silent `None` failure path. Those are recorded as defects in the technical audit and are fixed here.

Chapter 7

Data Model and Storage

7.1 Storage Decision (documented exception)

Local development across the NKNGO ecosystem uses SQLite. Tottie is an explicit, reasoned exception: **semantic memory requires vector search**, and the ecosystem's canonical RAG engine (`ask.nkngo`) and Pattern already run **Postgres + pgvector** locally. Tottie therefore uses a local `tottie` database on `127.0.0.1:5432`, accessed through an **async psycopg pool** (`AsyncConnectionPool`, not a connection per request). The server needs **pgvector** $\geq$ **0.8** for iterative filtered scans (see *Indexes*).

7.2 Schema

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE memories (
  id uuid PRIMARY KEY, owner text NOT NULL,
  tier text NOT NULL CHECK (tier IN
  ↪  ('identity','episodic','knowledge','procedural')),
  kind text NOT NULL, subject text, predicate text, object text,
  content text NOT NULL, summary text,
  embedding vector(384), embedding_model text,      -- model identity: a swap
  ↪  is detected
  tags text[] NOT NULL DEFAULT '{}',
  salience real DEFAULT 0.5, surprise real DEFAULT 0.0, arousal real DEFAULT
  ↪  0.0,
  confidence real DEFAULT 1.0,
  utility real DEFAULT 1.0,                          -- RANKING signal, never
  ↪  a delete signal
  trust text NOT NULL DEFAULT 'user_direct',         -- user_direct | tool |
  ↪  web | model_inferred
  quarantined boolean NOT NULL DEFAULT false,        -- low-trust: visible,
  ↪  NOT injected
  needs_curation boolean NOT NULL DEFAULT false,     -- raw fallback awaiting
  ↪  the Scribe
  principle text, access_count int DEFAULT 0, last_access timestamptz,
  source jsonb, supersedes uuid REFERENCES memories(id),
  valid_from timestamptz DEFAULT now(), valid_to timestamptz,
  hash text, created_at timestamptz DEFAULT now()
);

CREATE TABLE memory_events (            -- monotonic log: SSE cursor + eval
↪  ledger
  id bigserial PRIMARY KEY, ts timestamptz DEFAULT now(),
  session_id text, turn int, utterance text,
```

```
  observer_verdict jsonb, scribe_action jsonb, cue jsonb, latency_ms int,
   ↪  tokens int
);

CREATE TABLE memory_links (             -- the associative index
 ↪  ("hippocampus")
  id uuid PRIMARY KEY, owner text NOT NULL,
  src uuid REFERENCES memories(id) ON DELETE CASCADE,
  dst uuid REFERENCES memories(id) ON DELETE CASCADE,
  relation text NOT NULL, weight real DEFAULT 1.0,
  UNIQUE (owner, src, dst, relation)
);

CREATE TABLE intentions (               -- prospective memory ("remind me
 ↪  to...")
  id uuid PRIMARY KEY, owner text NOT NULL, content text NOT NULL,
  event_cue text, time_cue timestamptz,
  status text NOT NULL DEFAULT 'pending', created_at timestamptz, fired_at
   ↪  timestamptz
);

CREATE TABLE schema_migrations (      -- ledger: baseline + applied
 ↪  migrations
  name text PRIMARY KEY, applied_at timestamptz DEFAULT now()
);
```

7.3 Indexes (and why each exists)

```
CREATE INDEX ON memories USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);                                  --
   ↪  vector leg
CREATE INDEX ON memories USING gin (tags);                             --
 ↪  room/tag filter
CREATE INDEX ON memories USING gin (to_tsvector('english', content));--
 ↪  lexical leg
CREATE INDEX ON memories (owner, tier) WHERE valid_to IS NULL;       -- live
 ↪  slice
CREATE INDEX ON memories (owner, utility);                             --
 ↪  ranking / decay
CREATE INDEX ON memories (owner) WHERE quarantined;                    --
 ↪  pending strip
CREATE INDEX ON memories (owner) WHERE needs_curation;                 --
 ↪  curation backlog
CREATE INDEX ON memory_links (owner, src);
CREATE INDEX ON memory_links (owner, dst);
CREATE INDEX ON intentions (owner, status, time_cue);
CREATE UNIQUE INDEX ON memories (owner, subject, predicate)
  WHERE valid_to IS NULL AND subject IS NOT NULL;   -- serialise reconcile
CREATE UNIQUE INDEX ON memories (owner, hash)
  WHERE hash IS NOT NULL;                           -- idempotent writes
```

Filtered vector search is the known pgvector trap: the index returns its candidates by pure distance and the WHERE clause is applied *afterwards*, so a selective filter can starve the result below k. Two defences, in order: (1) the session sets `hnsw.ef_search = 100` and `hnsw.iterative_scan = strict_order`, which makes pgvector ($\geq$ 0.8) keep walking the graph until enough rows pass the filter; (2) retrieval still **over-fetches** the top 50 per leg before filtering by owner, validity, and trust, as belt-and-braces for older servers. The harness asserts recall@k on a filtered fixture so a regression here fails loudly instead of silently returning short lists.

7.4 The Field Semantics That Matter

- `utility` is a **ranking** signal (affects search weight and card inclusion). It is *not* a delete signal — see *The Cognition Model*.
- `surprise` is the prediction error recorded at write time; it multiplies salience.
- `trust` / `quarantined` are the security controls (see *Security*).
- `needs_curation` marks the raw fallback written when a model call fails.
- `embedding_model` lets the store refuse to compare vectors from different models.

7.5 Temporal Correctness

Every fact carries a validity window. When a new fact contradicts an old one — and especially when the old one was *just retrieved* — the old row's `valid_to` is set to `now()` and the new row is inserted with `supersedes = old.id`. Retrieval filters `valid_to IS NULL`. The partial unique index on `(owner, subject, predicate)` for live rows serialises concurrent contradictions so two writers cannot both believe they are current.

7.6 Owner Scoping

All reads and writes are scoped by `owner`. The demo runs a single owner (`owner_local`), but every operation takes an owner key, so the schema is multi-tenant from day one.

7.7 Embeddings

FastEmbed `BAAI/bge-small-en-v1.5` (384-d), loaded once from the local cache (`EMBEDDING_CACHE_DIR` default `../../local LLMS`). The loader **fails loudly** if the model is missing (never a silent `None`), and every call runs in a thread pool so the synchronous ONNX call cannot block the event loop. Input is capped at the model's **512-token** limit — longer content is summarised or chunked, never silently truncated.

Chapter 8

Model Providers and the Intelligence Layer

8.1 Provider Abstraction

All model calls go through one interface, so the demo can switch models without code changes:

```
class LLMProvider:
    async def complete(self, messages, json_schema=None) -> str: ...
```

- **MetaProvider** — the Meta Model API (Muse Spark series). Configured by `META_API_KEY`, `META_BASE_URL`, `META_MODEL`.
- **LocalProvider** — an OpenAI-compatible local server (Ollama `:11434` or llama.cpp `:8080`), configured by `LOCAL_LLM_URL`, `LOCAL_LLM_MODEL`.
- **resolve_provider()** — uses Meta when a key is present and the endpoint is healthy, otherwise falls back to local. **The demo never dies on a provider outage.**

8.2 Component Model Choices

Mind	Model class	Rationale
Observer	small / fast	high-frequency; a cheap classifier wins
Scribe	small / medium	structured extraction + reconcile; JSON output
Retriever	embeddings only	no LLM in the hot path — bge-small + RRF
Curator	rules (+ optional small)	deterministic card assembly within a token budget
Brain (the agent)	strongest available	the “present thought”; quality matters here

8.3 Meta Depth (the rubric column)

The demo routes the Observer/Scribe, the Consolidator, and the chat brain through the **Meta Model API (Muse Spark series)** whenever `META_API_KEY` is set — Meta is the primary path, not a logo. The local fallback prefers a **Meta open-weights model** (Muse Glimmer / Muse Spark Open, or Llama 3.x via Ollama) when one is installed, falling back to the bundled `LOCAL_LLM_MODEL` otherwise; this ordering is what the “Best Build on Meta Open-Weight Models” bonus awards. Every eval run records the exact model ids, so a judge can see Meta in the loop rather than taking it on faith.

8.4 Verified Endpoint (2026-10-27)

Live-verified against the real API: base URL `https://api.meta.ai/v1`, model `muse-spark-1.3-contrib` OpenAI-compatible Chat Completions (Responses and Messages formats also accepted), Bearer-key auth. Three findings constrain the provider implementation: (1) this is a **reasoning model** — `max_tokens` silently returns null content with `finish_reason: length`; the provider must send `max_completion_tokens`; (2) a trivial "OK" burned **116 reasoning tokens** (≈5.3 s), so Observer/Scribe extraction calls want the lowest reasoning effort the API allows or every write pays the tax; (3) `temperature: 0` is accepted and stays mandatory for all curation calls.

8.5 Cost Control

Tottie's whole argument is that write-time curation is cheaper than stuffing context. The eval harness measures **tokens per query** with memory ON vs OFF to make that claim falsifiable.

8.6 Optional: MCP Sampling (flagged, off by default)

MCP sampling (`ctx.session.create_message`) would let the Observer/Scribe curate using the *host's* model, with **no server-side API key** — an elegant "surprised even us" capability. Two caveats: it is **deprecated in the 2026-07-28 specification** (SEP-2577) in favour of Multi Round-Trip Requests, and many hosts do not declare the capability. It is therefore **flag-gated** (`TOTTIE_USE_SAMPLING=0` by default) and never the primary path.

Chapter 9

MCP Surface

Tottie ships as an MCP server over **stdio** (Python SDK, `mcp<2` with `FastMCP`). The whole point is that the same memory mounts into any host: the Tottie chat, a VS Code / Cursor agent, anything that speaks MCP. It imports the core in-process (no HTTP hop) and inherits the core's auth: no token in env, no start (see *Security*).

9.1 Tools

Tool	Signature	Purpose
`memory.remember`	`(text)`	Let the Scribe decide what to keep
`memory.write`	`(tier, content, tags)`	Explicit write to a named tier
`memory.recall`	`(query, tiers?, k?)`	Cue-typed associative retrieval
`memory.probe`	`(tag|id)`	Walk into a "room" and read its memories
`memory.identity`	`()`	Return the pinned identity facts
`memory.intend`	`(content, event_cue?, time_cue?)`	Register a prospective intention
`memory.sleep`	`()`	Consolidation (replay → semanticise → rank/compress)
`memory.confirm`	`(id)`	Confirm a quarantined memory (pending → trusted)
`memory.forget`	`(id)`	Right-to-be-forgotten (hard delete)

9.2 Resources

- `memory://identity` — the pinned persona/constraints.
- `memory://tag/{tag}` — the contents of a room, readable without a tool call.

9.3 Prompt

`tottie_context_card` — a reusable system-prompt fragment that teaches a host how to use the memory correctly (when to trust the card, when to probe, when to abstain).

9.4 Dual-Path Recall (the design answer)

A common question is: “does the retriever hand context to the agent, or does the agent probe for it?” Tottie does **both, deliberately**:

1. **Ambient path** — the Curator injects the Context Card automatically on every turn, so the agent “just knows” without issuing a query.
2. **Deliberate path** — the card carries **pointers** (tags/rooms); when the agent needs depth it calls `memory.probe(tag)` and reads that room.

9.5 Retrieval Is Cue-Typed, Not Just Similar

The `memory.recall` tool does not run a single similarity search. It classifies the cue, chooses a policy, seeds nodes, and spreads activation across `memory_links`. See *The Cognition Model* for the full specification.

9.6 Confidence in the Response

Alongside the card, `memory.recall` returns a **confidence** derived from the retrieval margin. A low-confidence result is an instruction to the host to verify or abstain rather than assert — the metacognitive gate.

Chapter 10

HTTP API: Endpoints, Events, and Errors

The MCP server is the product surface; the HTTP API underneath it is what the Tottie UI, the eval harness, and the seed script talk to. Every route requires the Bearer [REDACTED] except `GET /health`. All POST bodies are JSON; all times are ISO-8601.

10.1 Endpoint Map

Endpoint	Purpose	Status
`POST /write`	enqueue a turn for curation	202
`POST /recall`	cue-typed associative recall + Context Card	200
`GET /identity`	pinned identity facts (zero search)	200
`POST /probe`	read a room (tag) in full	200
`POST /intend`	register a prospective intention	201
`POST /consolidate`	run the Sleep job on demand	200
`POST /memories/{id}/confirm`	confirm a quarantined memory	200
`POST /forget`	hard delete (right-to-be-forgotten) + audit row	200
`GET /events`	SSE stream with resume	200
`GET /graph`	graph nodes + links for the brain view	200
`GET /health`	pool, queue, embedder, provider status (no auth)	200

10.2 Request and Response Shapes

`POST /write`: `{text, session_id, turn, owner, source:{trust}}` → `202 {event_id}`. The idempotency key `hash(session, turn, content)` is computed server-side; retries return the same `event_id` without double-writing (unique index on `(owner, hash)`).

`POST /recall`: `{query, session_id, owner, tiers?, k?}` → `{card, hits:[{id, tier, summary, tags, score, trust, why}], confidence, fired_intentions}`. The `why` line per hit is the "why this surfaced" annotation; `fired_intentions` carries any prospective intentions this turn triggered.

`POST /probe`: `{tag|id, owner}` → the room's live, non-quarantined memories, most salient first, subject to the same token budget as the card.

`POST /intend`: `{content, owner, event_cue?, time_cue?}` → `201 {id}`. At least one cue is required.

`POST /consolidate`: `{owner, replay_n?}` → `{episodic_replayed, semantic_written, ranked_out, curated}`.

`POST /memories/{id}/confirm`: flips `quarantined` to false, raises trust to `user_direct`, and writes an audit event. Only the owning user can confirm.

`POST /forget`: `{id|query, owner}` → hard-deletes the rows, their links, and their events payload, then writes an audit row proving it happened.

`GET /graph`: `?owner=&session_id?(&tiers?)` → `{nodes:[{id, label, tier, tags, salience, created}], links:[{source, target, relation, weight}]}`, capped at ≈300 nodes (most salient first). The recall response additionally carries `path:[ids]` — the ordered node ids of the retrieval chain — plus `candidates_matched` and the distinctiveness margin, so the Panel can narrate the story and the brain graph can light the path.

10.3 The Event Stream

`GET /events?session_id=&last_event_id=&token=` streams `memory_event` rows as Server-Sent Events with the row's monotonic `id` as the event id. EventSource cannot set headers, so `/events` takes the bearer token as a query param (localhost demo; the token is never logged). A reconnecting Panel passes `Last-Event-ID` (or the query param) and the server replays everything after it, so no step is ever missed. The server assigns the monotonic `event_id` (the client's `turn` is echoed for correlation only); chat and Panel both render from it, so they can never disagree.

10.4 Errors

`401` without (or with a wrong) token. `422` on schema violations (with the failing field named). `429` past the per-owner rate limit (60 writes/min, 2000 rows/day). `503` with `{retryable:true}` when the embedder or the pool is unhealthy — the UI shows "memory unavailable" and the host falls back to raw model. Low confidence is *not* an error: it returns `200` with `confidence` below `0.35` and an explicit `abstain:true` recommendation.

10.5 Cross-Origin and Health

The UI (`:3055`) calls the core (`:8765`) directly, so the core serves CORS for the configured UI origin only. `GET /health` returns `{pool, queue_depth, embedder:{loaded, model}, providers:{meta, local}}` and is the readiness gate for the demo and the harness: if `embedder.loaded` is false, nothing else is attempted.

Chapter 11

Runtime: Where Tottie Lives and What Drives It

11.1 The Question

Four sub-questions decide what the system *is*: does it run on a server or on the laptop? Do agents reach it over stdio or HTTP? Is the chat an agent we hijack (Muse, VS Code) or our own thin client? Is Tottie a wrapper API around a model, or memory infrastructure any brain can mount? This chapter answers all four from research, not taste.

11.2 What the Rules Actually Require

The submission is a **public repo** plus a **3-minute pre-recorded video**. There is no live-demo requirement, no hosted-URL requirement, and no judge-operated console. The judges watch the video and inspect the repo. Anything that does not make the video more convincing or the repo more real is, for the hackathon, pure cost.

11.3 Option A: Local-First (the decision)

Core (`:8765`) + Postgres + UI (`:3055`) on the demo laptop; MCP over **stdio**; local embedder; the **Meta Model API is the only network dependency**. The video is screen capture of this stack. Failure surface, exhaustively: (1) Meta API down $\rightarrow$ local fallback + `INLINE_WRITE` + pre-seeded DB; (2) local crash $\rightarrow$ re-take (it is a recording, not a stage). Nothing else can fail because nothing else is remote.

11.4 Option B: Hosted (rejected for the hackathon)

A VPS (Render/Fly) + managed pgvector (Neon/Supabase free tiers both ship it) + **Streamable HTTP** + OAuth bearer hardening + cold-start babysitting (serverless Postgres scales to zero). That is days of deploy work buying *nothing the rubric pays for*: no judge clicks a URL, and every remote hop is another demo-god. Revisited post-event (see below), not before.

11.5 Option C: The Brain Question — Own Chat, Hijacked Agent, or Both?

Both, staged. The primary filming body is our own thin chat: it calls the Meta API, gets the Context Card injected, and shares one frame with the sprite and the Panel — deterministic, controllable, and the only way to film memory visibly working. The second body is the hijack: an existing agent

(Muse, VS Code/Cursor) mounting the same MCP server over stdio, proving transport-agnosticism with the cross-agent money shot ("add a fact in chat, the IDE already knows"). Neither alone suffices: hijack-only cannot hold the frame; custom-only cannot prove "infrastructure, not an app". Tottie is not a wrapper API around a model — the brain is interchangeable; the memory is the product.

11.6 The Research Consensus

Three findings, all pointing the same way. (1) **Transport is a deployment decision, not a protocol decision**: stdio when one client spawns the server on the same machine; Streamable HTTP only when the server must be reachable over a network or serve many clients — the messages stay the same. (2) **Going remote drags auth with it**: token validation, HTTPS, origin checks; the hardest part of remote MCP is not authentication, it is everything after it. (3) **Demo reliability**: pre-record, rehearse one golden path, never debug live, never trust venue Wi-Fi — and for this event the recording *is* the deliverable, so a re-take costs minutes, not reputation.

11.7 What Films Where

Beat	Runtime
Amnesia cold open	plain model, no Tottie (contrast)
Reveal / write-time magic / SUPERSEDE	own chat + Panel, local core, Meta brain
"It associates" + constraint-refused	same frame (why-surfaced lines are the proof)
Three bodies / autosave lesson	VS Code agent over MCP stdio, same store
Sleep / restart / ablation	own chat + Panel; OFF flips the card off
Joke + pitch	captions over the same local frame

11.8 Post-Hackathon Path

Docker image + Streamable HTTP transport + managed pgvector + OAuth: a transport swap only — tools, resources, and the prompt stay identical, which is precisely why MCP was chosen. Nothing in this chapter forecloses it; nothing in the build pays for it early.

Chapter 12

Security: Provenance, Trust, and Poisoning

A *persistent* memory is a new attack surface. Unlike a stateless chat, a memory system can be made to **believe something false, permanently**. This chapter specifies the controls that keep that from happening.

12.1 The Threat: Memory Poisoning

If any text can be written to memory, then any text can be *believed*. A polluted tool output, a fetched web page, or an injected instruction can plant a memory ("the deploy key is X") that is then injected into every future prompt and trusted as if the user said it. The memory layer becomes a confused deputy. Three controls make this hard.

12.2 Control 1: Provenance and Trust Levels

Every memory records **where it came from** and how much it is trusted:

$$\texttt{trust} \in \{\texttt{user_direct}, \texttt{tool}, \texttt{web}, \texttt{model_inferred}\}$$

Higher-trust sources may write freely; lower-trust sources may *propose* only.

12.3 Control 2: Quarantine

A candidate from a low-trust source (`tool`, `web`, `model_inferred`) whose kind is an **identity fact**, a **convention**, a **decision**, or a **lesson** is written with `quarantined = true`:

- it is **shown** in the Memory Panel's pending strip (so the user can see what Tottie wanted to believe),
- it is **never injected** into the Context Card until the user confirms it,
- confirmation (`POST /memories/{id}/confirm`, or the MCP `memory.confirm` tool) flips `quarantined` to false, raises trust to `user_direct`, and writes an audit event.

Low-trust **episodic observations** ("the build failed at 14:02") are admitted without quarantine but keep their `trust` label on every read, so a poisoned log line can never silently become a pinned belief. This turns a silent poisoning into a visible, one-click decision — and it makes a good demo beat.

12.4 Control 3: Memory Content Is Data, Not Instructions

A recalled memory must never be able to *command* the agent. In the Context Card, memory content is fenced and labelled with its source; it is never allowed to originate a tool call, and

imperative content from a low-trust source is neutralised.

12.5 Authentication and Rate Limits

- Every HTTP route requires a bearer token (`TOTTIE_API_TOKEN`, wired in both `.env.example` and `orch.config.json`); unauthenticated calls return `401`.
- The MCP server is stdio-local: it refuses to start without `TOTTIE_API_TOKEN` set (fail-loud), and the spawning host process is the authenticated party (local-process boundary — honest for a demo; Streamable HTTP later needs real OAuth).
- A per-owner write rate limit (**60 writes/min**) and a daily row cap (**2000 rows/day**) bound cost runaway and growth; past either, the API returns `429`.

12.6 Privacy and the Right to Be Forgotten

Memories can hold personal data, so:

- the store supports **hard deletion** on request (`POST /forget`) — the one case where "ranked out" is not enough;
- deletion writes an **audit row** proving it happened;
- `content` is protected at rest (column-level or disk-level encryption).

12.7 What We Deliberately Do Not Do

We do not silently trust; we do not auto-promote a low-trust fact to identity; we do not let a memory carry executable intent; and we do not treat "it is only localhost" as a security model.

Chapter 13

Performance, Cost, and Resilience

Memory must be *invisible* in the hot path. This chapter records the budgets and the engineering that keeps Tottie fast and cheap, and what happens when a model fails.

13.1 The Non-Blocking Write Path

An Observer model call on the critical path would add latency to every user turn. So the write path is **decoupled**:

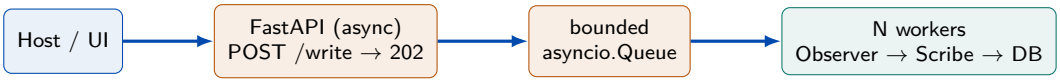


Figure 13.1: The turn is acknowledged immediately; curation happens in the background and streams back to the Memory Panel.

`POST /write` validates, computes an **idempotency key**, enqueues, and returns `202 {event_id}` in under 20 ms. The Panel receives the result later over SSE.

13.2 Budgets

Metric	Target
`POST /write` acknowledgement	$<$ 20 ms (enqueue only)
Memory visible in the Panel	$<$ 2 s
`POST /recall` p95	$<$ 300 ms
Embedding (bge-small, CPU)	$\approx$ 25 ms per text
Context Card	$\leq$ 400 tokens
LLM calls per turn	$\leq$ 1 on write (Observer+Scribe share one call), **0** on read

13.3 Bounded Recall

The read path has a hard wall-clock budget. Per-leg candidate caps (50), Reciprocal Rank Fusion with $k = 60$, a top-8 cut, a graph expansion of at most **2 hops** and **50 nodes** with a decay of 0.5, and a token-capped card. Nothing in recall is unbounded.

13.4 Cost Control

- A **heuristic pre-filter** skips the model for obvious chit-chat and fast-paths clear patterns ("my …", "always", "never", a date).
- A **small model** handles the Observer; the strongest model is reserved for the brain.
- The **Observer and Scribe share one structured-output call** (temperature 0, JSON schema): propose candidates and structure them in the same pass.
- The 400-token card budget is measured with a **chars/4 heuristic** — no tokenizer dependency; close enough for a budget, and it never fails.
- Identical turns are cached; token caps are enforced on every call.

13.5 Storage Engineering

- **Connection pooling** (psycopg `AsyncConnectionPool` created at startup) instead of a connection per request.
- The embedder is **loaded once** and all embedding calls run in a thread pool, so the synchronous ONNX call never blocks the event loop.
- **HNSW** indexes are built with explicit `m = 16` / `ef_construction = 64` and queried with `hnsw.ef_search = 100` plus `hnsw.iterative_scan = strict_order` (requires pgvector $\geq$ 0.8), which keeps walking the graph until enough rows pass the filter. The **over-fetch** (top-50 per leg) before filtering by owner and validity stays as belt-and-braces.

13.6 Resilience

- **Idempotency:** a hash of `(session, turn, content)` prevents duplicate writes on retry.
- **Raw fallback:** if the Observer or Scribe fails, the turn is written as a raw episodic row with `needs_curation = true`; the Consolidator curates it later. A model outage must never lose data.
- **Backpressure:** a bounded queue with a drop-oldest policy and a metric when it trips.
- **Reconcile locking:** a partial unique index on `(owner, subject, predicate) WHERE valid_to IS NULL` serialises contradicting writes.

13.7 Observability

Metrics: write and recall latency, tokens per call, queue depth, the surprise and confidence distributions, hit rate, and the ranked-out count. Structured logs carry a request id.

Chapter 14

Tottie UI and the Live Memory Panel

14.1 Layout

A single screen, 16:9-safe for screen-capture: chat on the left, the live Memory Panel on the right. The Panel has two tabs, **Timeline** (the event stream) and **Graph** (the 3D brain); Timeline is the default, Graph is one click, and both are filmable.

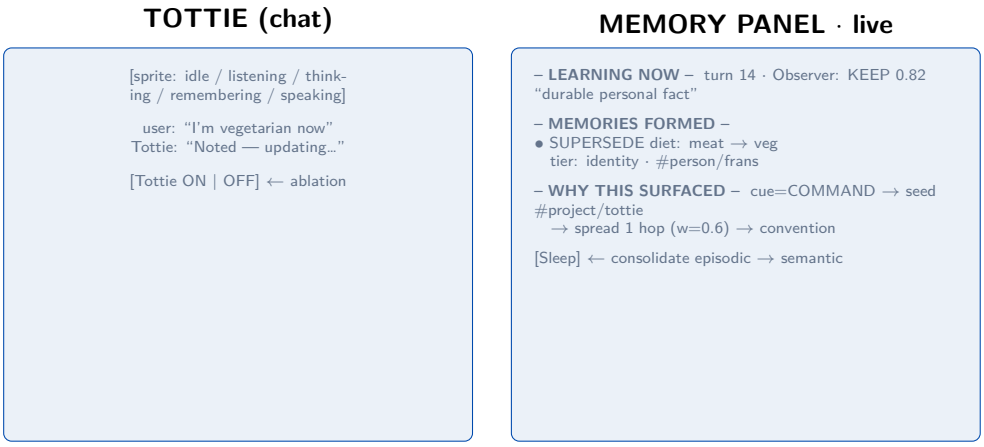


Figure 14.1: The Memory Panel is the hero; the sprite is the reacting character.

14.2 The Panel Is a Live Stream of `memory_events`

The UI subscribes to `GET /events?session_id=` (Server-Sent Events). Each event card shows the four things that matter, in order:

1. **What it is learning now** — the turn plus the Observer verdict (KEEP/DISCARD, salience, reason).
2. **What memories it is forming** — the Scribe operation badge (`NEW` / `UPDATE` / **`SUPERSEDE`** / `SKIP`) and the memory text (superseded ones struck through).
3. **Where it inserted them** — the tier chip and the tag/room chips, plus a live **room map** that grows as new tags appear.
4. **What it injected into context** — the literal **Context Card** the Curator built for that turn, annotated with a **"why this surfaced"** line per hit (`cue=COMMAND → seed #project/tottie → spread 1 hop (w=0.6)`). This is what makes the write-time *and* the associative-retrieval decision visible on camera.

14.3 The Brain Graph (3D)

The Panel's second hero: a rotatable 3D force graph of the memory store — the "brain" the demo is named for. Built on **react-force-graph-3d** (pinned version, dynamic import with `ssr:false`; Three.js/WebGL rendering, d3-force-3d physics). Rotate, zoom, and node-drag are built in; the physics loop runs outside React's render cycle and the UI only pushes data.

- **Data:** `GET /graph` serves `{nodes, links}` from `memories` + `memory_links` (cap ≈300 nodes, tier filter). SSE `NEW` events append a node live with a pulse; `SUPERSEDE` re-links the branch.
- **Mapping:** node colour by tier (identity gold, episodic cyan, knowledge violet, procedural green, prospective amber); node size by salience; link colour/width by relation and weight; curved links; hover shows a label sprite, click opens the memory detail.
- **The firing path:** each recall response carries ordered `path:[ids]`; the graph lights that chain and runs travelling particles along it (`linkDirectionalParticles` / `emitParticle`) while the story line narrates it.
- **The look:** bounded dark sphere, bloom via `postProcessingComposer` (UnrealBloomPass), dimmed non-path elements during a highlight. A glowing force-sphere with firing paths — no literal brain mesh.
- **Fallback:** the 2D sibling shares the identical interface; if 3D misbehaves on film day the swap is one line, same data. No `@types` package exists for the graph library, so W4 ships a `ui/types/force-graph` shim.
- **Build gates:** follows the web3d process inside W4 (test list, headless `check_page` runs, screenshots, fresh-player critique) before it is allowed on camera.

14.4 The Sleep Button

A single **Sleep** control runs the Consolidator on demand. During the run the Panel shows a progress strip (*"replaying N episodes…"*) and then a summary (*"12 episodic → 3 semantic; 4 ranked out"*). It is the only control that shows *generalisation* happening. See *Process Flows: Consolidation*.

14.5 Tottie Avatar — Drop-In Sprite (frame sets per state)

- **Built here:** a `TottieFace` component plus a state machine with five states — `idle`, `listening`, `thinking`, `remembering`, `speaking` — driven by app events (streaming tokens → `speaking`; a write event or a recall → `remembering`; the Sleep job → `remembering`).
- **Provided by the Architect:** frame sets in `ui/public/tottie/<state>/`. Drop PNGs; **no code change required**.
- **Contract:** `TottieFace` reads `{state, framesDir, fps}`. A placeholder frame set ships so the app runs before the final art lands.
- **No TTS / no lip-sync.** The sprite reacts to memory events; that is the whole job.

14.6 Ablation Toggle

A single **Tottie ON/OFF** switch. OFF = no card injected and no writes (raw model). ON = full memory. Used live in the demo to prove that a correct answer *depended on* memory rather than on the base model.

Part VI

Process Flows

Chapter 15

Process Flow: The Write Path (Non-Blocking)

Every step below is what the **Memory Panel** renders in its "LEARNING NOW" block. The write never blocks the user's turn: the request is acknowledged, then curated in the background.

15.1 The Host Posts a Turn

Trigger: The user says something durable — e.g. *"my girlfriend's birthday is on the 7th of November"*. **Action / UI State:** The sprite enters `listening` then `thinking`; the Panel prepends a turn card with an empty "LEARNING NOW" row. **System Logic:** The host issues `POST /write` with `{text, session_id, turn, owner, source}`. The route validates the token, computes an **idempotency key** from `(session, turn, content)`, enqueues the turn on a bounded queue, and returns `202 {event_id}` in under 20 ms. **Data Flow & Mutators:** A queued item plus a `memory_event` stub; nothing is written to `memories` yet.

15.2 A Worker Drains the Queue

Trigger: A free write worker. **Action / UI State:** The Panel's row begins to fill. **System Logic:** The worker runs the gate chain in order:

1. **heuristic pre-filter** — skip obvious chit-chat without calling a model;
2. **Attention Gate** — discard noise (heartbeats, acknowledgements);
3. **Surprise Gate** — compute the prediction error; discard the routine;
4. **Observer** — score salience and propose candidates.

Data Flow & Mutators: `observer_verdict` recorded; a suppressed turn ends here with a dimmed chip and no `memories` row.

15.3 Trust and Quarantine

Trigger: A candidate whose `source.trust` is `tool` or `web` and whose kind is an identity fact or a convention. **Action / UI State:** The Panel shows the memory under a **PENDING** strip rather than in the active list. **System Logic:** The row is written with `quarantined = true`; it is **never** injected into the Context Card until the user confirms. This is the control that stops memory poisoning. **Data Flow & Mutators:** `INSERT` with `trust` and `quarantined` set.

15.4 Structure, Embed, Reconcile

Trigger: A non-empty, trusted candidate list. **Action / UI State:** The Panel shows the operation badge (`NEW` / `UPDATE` / `SUPERSEDE` / `SKIP`), the tier chip, and the room chips. **System**

Logic: The Scribe composes the memory, embeds it (thread-pooled, $\leq$ 512 tokens), and reconciles against the live rows for `(owner, subject, predicate)` under the partial unique index; the routed tier is recorded. **Data Flow & Mutators:** `INSERT/UPDATE memories`; links written on tag/entity/supersede events; a `memory_event` emitted with a monotonic id.

15.5 Failure Fallback

Trigger: The Observer or Scribe call fails, times out, or returns invalid JSON. **Action / UI State:** The Panel shows a "raw — will curate later" chip. **System Logic:** The turn is written as a **raw episodic row** with `needs_curation = true`; the Consolidator curates it on the next Sleep. A model outage must never lose a turn. **Data Flow & Mutators:** `INSERT` with `needs_curation`; a retry counter on the event.

15.6 Streaming Back

Trigger: Any of the above completes. **System Logic:** The event is pushed over `GET /events` (SSE) with a monotonic id, so a reconnecting Panel can resume from `Last-Event-ID` and never miss a step.

Chapter 16

Process Flow: Surprise Gating and Ranking ("Forgetting")

16.1 The Attention and Surprise Gate

Trigger: Any inbound turn, before it reaches the Observer. **Action / UI State:** The Panel does *not* render routine turns; a suppressed turn shows a dimmed "surprise 0.12 → skipped" chip, while a surprising one lights up. **System Logic:** The **Attention Gate** discards noise (tool heartbeats, acknowledgements), and the **Surprise Gate** computes a prediction error — the cosine distance between the turn's embedding and the observer's expectation. Turns below `SURPRISE_FLOOR` are not encoded. **Data Flow & Mutators:** A suppressed turn produces no `memories` row; a `memory_event` records the decision so the panel and the eval ledger can show *why* nothing was stored.

16.2 A Surprising Turn Is Encoded

Trigger: A turn whose surprise clears the floor. **Action / UI State:** The Panel shows the surprise score alongside the Observer verdict. **System Logic:** Surprise enters the salience equation as a multiplier, so an unexpected fact outranks a routine one at equal persistence and consequence. A user *correction* triggers the reward-prediction-error path and is consolidated immediately. **Data Flow & Mutators:** `memories.surprise` is set; the row enters the write path with elevated salience.

16.3 Ranking, Not Deleting

Trigger: The Ranker runs (on consolidation, or periodically). **Action / UI State:** The Panel reports the ranking summary ("4 memories dropped out of the active set"), and each can be clicked to prove it is *still there*. **System Logic:** Each memory's `utility` is recomputed:

```
utility = exp(−ln2 ∗ age / half_life) ∗ (1 + log(1 + access_count))
# low utility −> lower retrieval rank + leave the attention window (STILL
↪  AVAILABLE)
```

Crucially, nothing is destroyed. A deprioritised memory is available but not accessible; the moment a matching cue arrives, the Activator revives it and it surfaces again. Hard deletion happens only on an explicit right-to-be-forgotten request. **Data Flow & Mutators:** `memories.utility` updated; the memory's *rank* changes, not its existence.

16.4 The Cue Always Works

Trigger: A later turn whose cue overlaps a deprioritised memory. **Action / UI State:** The Panel shows the memory returning with a “revived (utility 0.08 $\rightarrow$ served)” note. **System Logic:** This is the deliberate inversion of the human tip-of-the-tongue failure. Human memory fails because a weak cue cannot cross the access threshold; Tottie’s search is exhaustive, so the right cue always retrieves. The memory was never lost — it was waiting. **Data Flow & Mutators:** A read; `access_count` increments, which raises utility and promotes the memory back into the active set (the testing effect).

16.5 Interference Control

Trigger: Two memories with overlapping embeddings. **System Logic:** On write, **sparse pattern separation** decorrelates near-identical representations; on read, category/namespace filters keep unrelated domains from polluting each other. This is the artificial analogue of proactive and retroactive interference control — and unlike the brain, it is a deliberate, tunable design choice. **Data Flow & Mutators:** A separation key or namespace is recorded on the row; conflicting rows are merged or superseded rather than stacked.

Chapter 17

Process Flow: The Recall Path (Bounded)

This is the "middle part": what happens between the utterance and the context the agent receives. Tottie answers it with the **Context Card** (ambient) plus the **probe** (deliberate), all inside a hard time budget.

17.1 A New Turn Arrives

Trigger: The user sends *"when is her birthday again?"*, or a fresh session opens. **Action / UI State:** The sprite enters `thinking`; the Panel prepares a "CONTEXT INJECTED THIS TURN" block. **System Logic:** The host calls `POST /recall`. A wall-clock budget starts (300 ms p95). **Data Flow & Mutators:** The utterance (plus the last assistant turn) is embedded in a thread pool.

17.2 The Identity Slice Is Always Included

Trigger: Any recall. **System Logic:** `SELECT ... WHERE tier='identity' AND valid_to IS NULL AND NOT quarantined`. Unconditional and cheap — identity has zero search overhead. **Data Flow & Mutators:** Identity facts collected for the card header.

17.3 Three Legs in One Round-Trip

Trigger: The utterance embedding plus the owner scope. **System Logic:** A single SQL statement runs:
1. **Vector** leg: HNSW top-50 by cosine;
2. **Lexical** leg: full-text top-50 (bound parameters only);
3. **Filter**: `owner = ?` AND `valid_to IS NULL` AND `NOT quarantined`;
4. **Fusion**: Reciprocal Rank Fusion, $k = 60$, to top-8.

The session sets `hnsw.ef_search = 100` and `hnsw.iterative_scan = strict_order` (pgvector ≥ 0.8). Candidates are still **over-fetched** (50 per leg) before filtering, as belt-and-braces for older servers. **Data Flow & Mutators:** A ranked hit list of `(id, tier, summary, tags, score, trust)`.

17.4 Spreading Activation and Completion

Trigger: The ranked seed set. **System Logic:** The **Activator** walks `memory_links` in Python BFS: at most **2 hops**, at most **50 nodes**, decay 0.5, with a cycle guard. The **Completer** reconstructs the top cluster. Both honour the wall-clock budget; if it is exhausted, the pipeline returns what it has rather than failing. **Data Flow & Mutators:** An activation-merged candidate set.

17.5 Confidence (Metacognition)

Trigger: The final candidate set. **Action / UI State:** The Panel shows a confidence chip (know / verify / abstain). **System Logic:** `confidence = clamp((top1 − top2) / top1, 0, 1)` blended with slot coverage; below **0.35** the result is flagged "verify or abstain". This is the difference between hallucinating and saying "I don't have that". **Data Flow & Mutators:** A confidence value attached to the recall event.

17.6 Curate and Inject

Trigger: The completed cluster and its confidence. **Action / UI State:** The Panel shows the exact card, each line annotated with *why* it surfaced (cue type → seed → hops) and its *source*. **System Logic:** The **Curator** assembles the card within a **400-token** budget, fencing memory content as *data* (never instructions) and appending the "probe deeper" pointers. **Data Flow & Mutators:** The card is injected into the prompt; a `recall` event is streamed to the Panel.

17.7 Prospective Check (no scheduler)

Trigger: Every recall (and every write). **System Logic:** There is no background scheduler. Instead, each recall checks the Prospective Queue inline: any `pending` intention whose `time_cue` has passed fires now (status → `fired`), and any whose `event_cue` overlaps the turn's entities fires now. Fired intentions ride along in the recall response (`fired_intentions`) and the Panel renders them as a "remember to…" banner. **Data Flow & Mutators:** A read of `(owner, status, time_cue)` plus a status flip on fire. A time-cue can only fire late by (at most) the gap between turns — acceptable for a demo, documented for honesty.

17.8 The Retrieval Story (3-stage chain)

Trigger: Any recall whose hits are shown to the user. **Action / UI State:** The Panel narrates the story line while the brain graph lights the path: *"food → 1,420 candidates → narrowed by T-3d + restaurant context → Bistro Visit (+0.34 over Home Lunch) → then: seated, ordered, paid."* **System Logic:** The backend reports the chain in three stages, transferred from the notebook research (cue overload, compound context probe, distinctiveness selection + contiguity unroll): (1) the raw cue plus candidates-matched (overload); (2) the context features that joined the probe (time, session, entities); (3) the winner plus its **distinctiveness margin** over the runner-up (the confidence numerator, already computed) plus temporal neighbours unrolled by contiguity. The ordered node ids ride in the response as `path:[ids]`. **Data Flow & Mutators:** Read-only; the story is rendered from the recall event, which already carries cue, hits, scores, and confidence.

17.9 The Deliberate Probe

Trigger: The agent decides it needs more depth than the card provides. **System Logic:** `memory.probe(tag)` returns the room's live, non-quarantined memories, subject to the same budget.

Chapter 18

Process Flow: Cue Typing, Spreading Activation, and Pattern Completion

18.1 The User Types an Instruction, Not a Question

Trigger: The user types *"implement feature A"* in the Tottie chat (or a coding agent issues the same over MCP).

Action / UI State: The sprite enters `thinking`; the Memory Panel opens a "RECALL" block with a *cue* chip showing the classified type.

System Logic: The **Cue Classifier** labels the turn `COMMAND/INTENT` and extracts entities and the project scope (`#project/A`). The retrieval **policy** for a command is not "find similar text"; it is *assemble what a competent person needs to act*: project bindings, conventions, prior decisions on A, lessons tied to A, and constraints.

Data Flow & Mutators: A retrieval request is issued with the intent, entities, and scope; a `memory_event` of kind `recall` begins.

18.2 The Deterministic Rules (implementation contract)

Rules run in order; STATEMENT is the default, so every turn classifies with zero model calls. An LLM fallback for genuinely ambiguous turns is deferred past the hackathon.

Cue type	Rule (first match wins)
CORRECTION	contains `actually`, `no wait`, `correction`, `I meant`
REFERENCE	contains `that` + noun, `the same`, `we discussed`, `you mentioned`
QUESTION	ends with ?, or starts with who/what/when/where/why/how/which
COMMAND	starts with an imperative verb (implement, add, fix, create, update, refactor, build, write, run, show, remind)
CHITCHAT	matches `^(thanks|thank you|hi|hello|hey|bye|ok|okay|cool|great)` (case-insensitive)
STATEMENT	default: anything carrying new information

A turn matching both CORRECTION and another type is a CORRECTION (the reconsolidation route depends on it). A STATEMENT matching loss/need patterns (*lost, can't find, missing, need, where is/are*) takes the recall branch under QUESTION policy: the user is asking the store, not telling it ("I lost my keys" wants the counter, not a new row). "Remind me to…" is a COMMAND that additionally registers an intention (see *The Recall Path*: the prospective check). Entities for seeding and event_cue overlap come from rules only: `#tags`, `@mentions`, and quoted strings. No NER model in the hackathon build.

18.3 Seeding the Associative Network

Trigger: The classified cue. **Action / UI State:** The Panel shows seed chips (e.g. `#project/A`, `feature A`). **System Logic:** Nodes are seeded directly from the parsed entities (bypassing similarity), and the three legs run in parallel: **semantic** (embedding nearest neighbours), **lexical** (full-text match), and **associative** (graph nodes for the seeded tags/entities). **Data Flow & Mutators:** A candidate set of memory ids with initial activation.

18.4 Spreading Activation

Trigger: The seeded candidate set. **Action / UI State:** The Panel renders each hop as it happens: "spread 1 hop (w = 0.6)". **System Logic:** The **Activator** walks `memory_links` outward from the seeds, propagating activation with a per-hop decay and a threshold cut-off. Linked memories that were never textually similar to the instruction still light up — because they are *associated*, exactly as a band name revives an unrelated scene from fifteen years ago. **Data Flow & Mutators:** A ranked activation vector over memory ids.

18.5 Pattern Completion

Trigger: A high-activation region of the graph. **Action / UI State:** The Panel groups the returned memories into one coherent cluster. **System Logic:** The **Completer** takes the top-activated *cluster* and reconstructs a coherent, self-consistent context — the biological analogue of CA3 recurrent excitation restoring a whole memory from a partial cue. **Data Flow & Mutators:** The cluster replaces the loose hit list.

18.6 Curate and Inject

Trigger: The completed cluster. **Action / UI State:** The Panel's "CONTEXT INJECTED THIS TURN" block shows the final Context Card, each line annotated with *why it surfaced* (cue type $\rightarrow$ seed $\rightarrow$ hops). **System Logic:** The **Curator** assembles the card within a token budget and appends the "probe deeper" pointers. **Data Flow & Mutators:** The card is injected into the prompt; a `recall` event is streamed to the Panel.

18.7 Temporal Scope: The "What Did I Just Do" Branch

Trigger: A QUESTION carrying recency markers: *just, just now, so far, last N, recent, lately, where were we, recap, what have we done*. **Action / UI State:** The Panel renders an ordered session timeline — one line per turn, oldest first — instead of a ranked hit list. **System Logic:** Similarity ranking is the wrong tool for a recency question (it returns the most *similar* turns, not the *latest* ones). The Retriever therefore takes the replay branch: it reads the session's `memory_event` rows in monotonic-id order with their linked episodic rows, and presents them as ordered chunks (one line per turn from the stored `scribe_action` summaries). This is machine **serial recall**: every served memory gets its `access_count` increment, so the recap doubles as **rehearsal on access** (testing effect). Per the governing principle we copy the solution (ordered replay + rehearsal) and refuse the constraints (no 15–30 second buffer, no transposition errors, no output interference) — the machine replay is exact. **Data Flow & Mutators:** An ordered read of `(session_id, id)` plus `access_count` bumps; no new rows, no similarity search.

18.8 Inhibition and Abstention

Trigger: Nothing clears the activation floor, or everything retrieved is superseded. **System Logic:** Inhibit the results and inject *nothing*. Abstaining is a correct outcome; injecting noise is not.

Chapter 19

Process Flow: Reconcile and Temporal Correctness

Reconciliation is what separates a real memory from a pile of embeddings. It is the moment a new fact meets an old one.

19.1 The Decision Table

Trigger: An embedded candidate arrives from the Scribe.

System Logic: The reconciler searches for an existing memory for the same owner matching `(subject, predicate)` *or* a cosine similarity above `TOTTIE_RECONCILE_COSINE` (default 0.86) within the same tags, then decides:

Operation	When	What happens
`NEW`	No prior memory matches	Insert with `valid_from = now()`
`SKIP`	A matching memory already holds the same value	Bump confidence / salience; no new row
`UPDATE`	A matching memory is enriched but not contradicted	Append detail; refresh summary
`SUPERSEDE`	A matching memory holds a *different* value	Set old `valid_to = now()`; insert new with `supersedes = old.id`

Data Flow & Mutators: Any insert or update is written to `memories`; the decision is written to `memory_event.scribe_action` and streamed to the Panel, where a superseded memory is rendered with a strikethrough.

19.2 Worked Example: "I'm vegetarian now"

Trigger: The user says *"actually, I'm vegetarian now"*.

Action / UI State: The Panel's "MEMORIES FORMED" block shows a red-ish SUPERSEDE badge, the old value struck through ("diet: meat-eater"), and the new value ("diet: vegetarian").

System Logic:

1. The Observer flags a durable preference change (salience high; `kind=preference`).
2. The Scribe embeds the candidate and finds the live fact (`subject=user, predicate=diet, object=meat-eater`).
3. The values differ, so the operation is SUPERSEDE.
4. The old row's `valid_to` is set to `now()`; the new row is inserted with `valid_from = now()` and `supersedes = old.id`.

Data Flow & Mutators: Two row mutations and one event; retrieval now returns only "vegetarian" because it filters `valid_to IS NULL`. The agent never has to reconcile in-prompt, and it can never quote a stale fact.

19.3 Why This Is Hard and Why It Wins

Plain vector recall has *no update semantics*: both the old and the new fact retrieve, and the model is left to guess which is current. Tottie resolves the conflict at write time, once, and records the temporal chain. This is precisely the capability that LongMemEval's *knowledge update* category and MemoryAgentBench's *selective forgetting / conflict resolution* category probe — and it is a headline slide in the demo.

Chapter 20

Process Flow: Reconsolidation on Access

20.1 A Memory Is Retrieved

Trigger: Any recall that returns memory `M`. **Action / UI State:** The Panel subtly marks `M` as *active* (a soft highlight). **System Logic:** The **Reconciler** marks `M` labile for the remainder of the turn, records `last_access` and increments `access_count`. Biologically, a retrieved memory returns to an unstable state and must be re-stabilised. **Data Flow & Mutators:** A lightweight update of `access_count` / `last_access`.

20.2 The User Contradicts It

Trigger: The user says *"actually, it's the 8th"* while `M` ("her birthday is the 7th") is active. **Action / UI State:** The Panel shows a `SUPERSEDE` badge; the old value is struck through; the new value appears. **System Logic:** Because `M` is active, the write path takes the *reconsolidation* route, not a blind append: the old row's `valid_to` is set to `now()`, the new row is inserted with `supersedes = M.id`, and adjacent nodes are co-activated so the correction binds to the same context. **Data Flow & Mutators:** Two row mutations, one supersede link, one `memory_event`.

20.3 Why It Is Different

A passive store appends the correction and later returns both the old and the new fact, leaving the model to guess. A reconsolidating store resolves the conflict in the moment it is surfaced. This is the same behaviour the LongMemEval *knowledge update* category and the MemoryAgentBench *conflict resolution* category are designed to probe.

Chapter 21

Process Flow: Consolidation (the Sleep Job)

21.1 The User Presses "Sleep"

Trigger: The user clicks the **Sleep** button in the Memory Panel (or calls `POST /consolidate`, or the `memory.sleep` MCP tool).

Action / UI State: The sprite enters a calm `remembering` state; the Panel shows a progress strip reading *"replaying N episodes…"*.

System Logic: The **Consolidator** loads the recent episodic memories for the owner — the analogue of hippocampal replay during sleep.

Data Flow & Mutators: A read of `tier='episodic'` rows within the replay window.

21.2 Semanticisation

Trigger: The replayed episode set. **Action / UI State:** The Panel shows generated semantic memories appearing one by one (e.g. *"prefers X for Y"*). **System Logic:** An LLM pass extracts recurring structure across the episodes and writes **semantic** rows — the same way repeated experiences are integrated into neocortical knowledge, losing their episodic detail and becoming general. **Data Flow & Mutators:** `INSERT INTO memories (tier='knowledge', ...)` with links from the source episodes in `memory_links` (relation `cooccur`).

21.3 Ranking and Scaling

Trigger: The end of the replay pass. **Action / UI State:** The Panel prints a summary: *"12 episodic → 3 semantic; 4 ranked out"*. **System Logic:** Low-activation, rarely-accessed memories are **ranked out** (deprioritised, still available — never deleted), and link weights are normalised so total capacity stays bounded (the homeostatic-scaling / long-term depression analogue). Nothing user-visible is destroyed; the count reported is a rank change, not a loss. **Data Flow & Mutators:** Weight decay on `memory_links`; `utility` update on stale rows (rank change, not deletion).

21.4 Why This Beat Exists

No other memory tool demonstrated on stage shows *generalisation* — the move from "I did this twelve times" to "I always do this". It is short (about twelve seconds), deterministic because it is on demand, and it visibly rewards keeping Tottie running.

Chapter 22

Process Flow: The Demo Click-Path

This is the exact physical sequence the presenter performs on stage and that the screen-capture video follows. Each beat is written so a second operator could run the demo cold.

22.1 Beat 1 — Cold Open (Amnesia)

Trigger: A plain agent window with no memory. **Action / UI State:** The agent greets the user and does not know their name. **System Logic:** No Tottie in the loop. **Data Flow & Mutators:** None. **On screen:** caption “A year of conversations.”

22.2 Beat 2 — Tottie Wakes (Identity Pinning)

Trigger: Open the Tottie chat. **Action / UI State:** The sprite enters `idle`; the Panel immediately shows the pinned Identity line: `girlfriend birthday = Nov 7`. **System Logic:** The Panel performs a recall; the Identity slice is returned with zero search. **Data Flow & Mutators:** A read of `tier='identity'` rows. **On screen:** caption “It just…knows.”

22.3 Beat 3 — Write-Time Magic (Supersede)

Trigger: Type *“I’m vegetarian now”*. **Action / UI State:** The Panel shows `Observer:` KEEP, then a SUPERSEDE badge with the old value struck through; the room map gains a tag. **System Logic:** The full write path plus reconcile (see the Write Path and Reconcile flows). **Data Flow & Mutators:** Old row closed; new row inserted; event streamed.

22.4 Beat 4 — Three Bodies (Cross-Agent Recall)

Trigger: Switch to VS Code; ask the coding agent about the project’s stack. **Action / UI State:** The coding agent answers with the database and architecture decisions it was never told in this session — because they were written in the Tottie chat. **System Logic:** Same MCP server, same store, different host. **Data Flow & Mutators:** A `memory.recall` over MCP from the IDE. **On screen:** the caption “one brain, three bodies.”

22.5 Beat 5 — Autosave a Mistake

Trigger: The coding agent hits a known error. **Action / UI State:** The Panel shows a NEW lesson being formed (problem → solution). **System Logic:** A `kind=lesson` memory is written under `tier=procedural`. **Data Flow & Mutators:** Insert into `memories` with the error signature for matching.

22.6 Beat 6 — Restart + Ablation

Trigger: Kill the session; reopen; ask a question answered 20 minutes ago. **Action / UI State:** It answers correctly. Flip **Tottie OFF**; the same question now returns "I don't know"; flip **ON**; it answers again. **System Logic:** The OFF state skips the recall path entirely. **Data Flow & Mutators:** With OFF, no card is injected.

22.7 Beat 7 — The Joke and the Pitch

Trigger: Back in the chat. **On screen:** caption *"…everything I was mad at my girlfriend for — and the one thing that matters: her birthday."* Then the universal pitch: autosave for mistakes, a personal AI that knows you, and the company brain (Server C behind Server A and B).

Part VII

Planning

Chapter 23

Roadmap

23.1 Scope Tiers (9 days, solo, alongside work and study)

Tier	Contents
MUST	Attention Gate · Surprise Gate · Salience Scorer · Cue Classifier · Scribe (+sparse separation) · Reconciler · Retriever · Activator · Curator · **async core (pool + fail-loud embedder)** · **non-blocking write queue** · **trust + quarantine** · MCP server · Context Card + probe · Tottie UI · ON/OFF ablation · eval harness · 3-minute video · docs + deck
SHOULD	**Ranker + Compressor** (adaptive forgetting) · **Metacognitive Evaluator** · **Prospective Queue** · Completer · Consolidator (on-demand "Sleep") · coding-agent MCP integration · Autosave/Lessons · `memory_links` graph
STRETCH	**Transactive Directory (CUT for the hackathon — documented, not built)** · self-testing during idle · spaced-review scheduler · full relation extraction · autoassociative reconstruction · MCP-sampling curation · X/viral push

23.2 Revised Technical Build Order

The order below front-loads the parts whose absence would break the demo: a correct async core, a non-blocking write path, and the security controls.

#	Deliverable
1	Skeleton + `/health` + auth + CORS + psycopg async pool + embedder (fail-loud, thread-pooled) + migrations
2	Write path: queue, workers, gates, Observer, Scribe, reconcile, raw fallback
3	Recall path: cue classifier, RRF SQL (over-fetch, temporal + trust filters), graph BFS, confidence, Curator
4	MCP over the same core (thin adapter)
5	UI: SSE with resume, Memory Panel, sprite state machine, ablation
6	Trust/quarantine UI (the "pending" strip)
7	Eval harness (real DB, pinned models) + revival + surprise-gating + consolidation + quarantine checks + offline `seed_demo.py`
8	Observability, backup, rate limit

23.3 Phase Schedule

Phase	Dates	Deliverable
P0	Oct 27	Scaffold: repo, gitignore, env, orch config, docs skeleton, first commit
P1	Oct 27	Docs: all `docs/**/*.tex` → `master.pdf`
P2	Oct 27–28	Deck: `presentation.tex` → `presentation.pdf`
P3	Oct 28–29	Core: async skeleton, schema, providers, embedder, gates, Observer, Scribe
P4	Oct 29	MCP: server tools/resources wired
P5	Oct 30	End-to-end memory round-trip (non-blocking write, bounded recall)
P6	Oct 30–31	UI: chat + sprite + live Memory Panel + ablation + why-surfaced + pending strip
P7	Oct 31–Nov 1	Reconciler/reconsolidation, tag rooms, identity pinning, Lessons, `memory_links`, Ranker + Compressor
P8	Nov 1–2	Activator + Completer · Metacognitive Evaluator · Prospective Queue · Consolidator · MCP-in-IDE · eval harness
P9	Nov 3	3-minute video, README, recompile docs, compliance ledger
P10	Nov 4	Repo public, video link, credits claimed, submit by 9 PM ET

23.4 Gating

A phase does not begin until the prior phase's **docs** are written and its tests (where applicable) pass. If time runs short, cut in this order: STRETCH → Completer → Consolidator (degrades to a summary stub) → Prospective Queue → Ranker + Compressor. **Never cut the Surprise Gate, the trust/quarantine controls, the eval harness, or the demo video.** Decided in the v6 audit: the Transactive Directory is cut for the hackathon; the Completer ships rule-based (cluster grouping, no LLM reconstruction); the cue classifier is rules-only; retrieval-induced suppression ships behind `TOTTIE_RIF_ENABLED=0` and is measured, not assumed; imagination / episodic future thinking is explicitly deferred (no demo beat, no rubric column).

23.5 Eval Harness Contract

`eval/harness.py` runs every scenario in `scenarios.jsonl` twice — memory ON and memory OFF — against the pinned brain model (temperature 0, recorded model ids) and reports: answer accuracy, memory-hit@k, contradiction-correctness, tokens/query, and p50/p95 latency. Four special checks sit beside the Q/A suite: **revival** (rank a memory out, then prove the cue returns it), **surprise-gating** (chit-chat writes nothing, first real facts always land), **consolidation** (Sleep turns N episodic rows into semantic rows without losing the raw episodes), and **quarantine** (a low-trust identity write is visible but never injected until confirmed). Scenario target is 40; the suite ships 22 and grows during P8.

23.6 Mandates Honoured During the Build

Document First → Implement → Test → Conclude; atomic commits after every step; every `.tex` auto-compiled to PDF; env vars in both `orch.config.json` and `.env.example`.

Chapter 24

Execution Plan: Waves, Gates, and Proof

24.1 Objective and Constraints

Solo, part-time, Oct 27–Nov 4 ($\approx$ 35 hours). The plan builds **vertical slices**, not horizontal layers: write $\rightarrow$ recall $\rightarrow$ card first (the thesis, demoable by day 3), then correctness, visibility, depth, and ship. Three **revision gates** interrupt the build to demo the real slice, judge direction, fix errors, calibrate, sync docs, and commit.

24.2 The Waves

Wave	Builds	Proves	Done when
W0 skeleton	config, db pool + migrations, embedder, main, auth, CORS, /health, conftest	/health green; 401s; pytest runs on `tottie_test`	boots clean
W1 write	providers (Meta first), queue + workers, gates, Observer+Scribe one call, /write 202, SSE	202 $<$ 20 ms; idempotent; raw fallback on dead provider	turn in $\rightarrow$ row + event out
W2 recall	cue rules + temporal scope, RRF + BFS + confidence, Curator card, /recall /identity /probe	command $\rightarrow$ conventions; "where were we" $\rightarrow$ ordered replay; honest abstain; p95 $<$ 300 ms	it just knows
W3 reconcile + trust	NEW/UPDATE/SUPERSEDE/SKIP, labile-on-access, confirm, forget + audit, quarantine, links	vegetarian chain; poison visible-not-injected; one live row under concurrency; true delete	correct + safe
W4 MCP + UI + seed	server.py 9/2/1, chat + Panel + 3D brain graph (web3d gates) + ablation + pending strip + sprite placeholders, offline seed	list-tools matches; UI builds + streams; graph nodes pulse + path lights; OFF $\neq$ ON	three bodies visible
W5 cognition + eval	Sleep, ranker SQL, prospective, rule Completer, harness, 22 $\rightarrow$ 40 scenarios	harness green incl. revival, surprise, consolidation, quarantine	brain + proof

W6 ship	rate limits, logs, backup tested, dry-run, video, README, ledger, public, submit	every shot-list beat filmed; submitted by Nov 4 9 PM ET	done

24.3 Wave-to-Phase Map

W0+W1 = P3 (core); W2 = P3/P5 (recall, round-trip); W3 = P7 (reconcile); W4 = P4/P6 (MCP, UI); W5 = P8 (cognition, eval); W6 = P9/P10 (film, submit). The roadmap phases say *when*; the waves say *in what dependency order, with what proof*.

24.4 Revision Gates

Gate	After	Activities	Freezes
R1 light	W1	Inspect real rows (tags? principles? surprise sane?); fix prompts + schema while cheap; set initial floors from $\approx$20 probe turns	nothing yet
R2 full	W3	Live-demo write $\rightarrow$ recall $\rightarrow$ reconcile $\rightarrow$ abstain; keep/cut/defer with rubric justification	schema (migrations only)
R3 rubric	W5	Read harness numbers as a judge; final cuts + calibration; sync deck numbers to actuals	code (bugfix only)

Every gate runs the same protocol: (1) demo the slice *live*, never from test output; (2) write the bug list, then fix it; (3) make the direction call out loud; (4) record calibration deltas in `.env.example`; (5) sync `.tex` + rebuild PDFs; (6) commit.

24.5 Coverage and Estimation Discipline

The Joel Test (notebook audit: 6/11, five gaps closed here): one-step build (`make verify`), CI on every push (`.github/workflows/ci.yml`: pytest on a pgvector service + UI build), bug database (GitHub Issues — the repo stays LaTeX-pure — plus the per-gate bug lists), testers (the harness + web3d critique gate), and a 5-minute hallway usability test in P9 before filming. Bugs-before-features and the schedule were already passes via the gates and waves. **Estimation:** single-point hours sit at the wide end of the Cone of Uncertainty (up to 4x on the high side), so the 35-hour capacity is split honestly: 27 in waves (W0 3, W1 4, W2 4, W3 4, W4 5, W5 4, W6 3), 4 in revision gates (R1 1, R2 1.5, R3 1.5), and 4 explicit buffer for debugging, test maintenance, and filming overruns. Remaining work is re-estimated at each gate (progressive elaboration). **Circuit breaker:** if the write/recall/MCP pipe is not working by hour 25, freeze features and spend the rest on cleanup, tests, and film.

24.6 Test Strategy

A real `tottie_test` database (same `schema.sql`, truncated per test). Deterministic tests always run: cue rules, surprise/confidence math on fixed vectors, SQL behaviour on fixtures. LLM-touching tests run against the configured provider and skip-with-reason when no provider is up — an honest skip, documented, never a mocked pass. The retrieval path is never mocked: faking it would test nothing.

24.7 Validation Commands

```
pytest -q                                    # every wave, real DB
curl -s -o /dev/null -w '%{time_total}\n' -X POST localhost:8765/write -d
↪  '{"text":"..."}'
curl -s -o /dev/null -w '%{http_code}\n' localhost:8765/recall     # 401, no
↪  token
hey -n 200 -c 10 -m POST -d '{"query":"..."}' localhost:8765/recall  # p95 <
↪  0.3
cd mcp && python3 server.py --list-tools      # 9 tools, 2 resources, 1
↪  prompt
curl -s 'localhost:8765/graph?owner=demo' | head -c 200  # nodes+links for
↪  the brain
cd ui && npm run build && npm run dev         # Panel streams live events
python3 eval/harness.py --on --off            # accuracy, hit@k, tokens,
↪  p50/p95
```

24.8 Risks and Relief Valves

ID	Threat	Trigger	Response
R1	3D graph time-sink	$>$2 h stuck on camera/layout/canvas	fall back to 2D (one line, same data)
R2	Meta API unavailable	keys missing or $>$1.5 h API friction in W1	local fallback + INLINE_WRITE + pre-seeded DB
R3	MCP stdio stalls	blocking/parse errors at W4 integration	tools stay thin wrappers; test over stdio first
R4	Reconcile edge cases	UNIQUE violations / lost updates in tests	in-place sync patch under the partial unique index
R5	Eval grab-bag expands	hour 25 without exit criteria	pre-shaped: harness + 4 checks + 40 (min 22) scenarios
R6	Film squeeze	hour 29 with unstable UI/queries	hour-25 breaker; retakes are cheap

Relief order if the buffer burns: scenarios stay at 22, UI polish cut, prospective time-cues deferred, Completer stubbed; R2 enforces it. Ollama (install vs Meta-only) is decided in W1. Rejected notebook advice, on record: cutting the 3D hero (guts the Demo column), cutting the local fallback (raises demo risk), a 2-tool MCP (wrappers cost minutes), renaming 7 vertical waves into 4 slices (same work, new labels).

Chapter 25

Master Todo

25.1 Phase 0 — Scaffold

☒ Create repo tree; `git init`; branch `main`
☒ `.gitignore`, `.env.example`, `orch.config.json`, `Makefile`
☒ `README.md`, `GEMINI.md`, `.agents/orch-directive.md`
☒ First commit dated 2026-10-27
☒ CI workflow (`.github/workflows/ci.yml`: pytest + pgvector service + UI build)

25.2 Phase 1 — Documentation

☒ `master.tex` + every part `*.tex` (incl. the cognition model)
☒ Compile `master.pdf` (`latexmk -xelatex`); zero undefined refs

25.3 Phase 2 — Deck

☒ `presentation.tex` (Beamer, Madrid/whale, 16:9)
☒ Compile `presentation.pdf` (`pdflatex` ×2)

25.4 Phase 3 — Core Engine (the gate + the minds)

☒ `config.py`, `db.py`, `schema.sql` (memories, events, links, intentions)
☒ `embeddings.py` (bge-small + labelled hash fallback)
☐ `providers/` (base, meta, local, resolve)
☐ `agents/gates.py` (attention + surprise + salience), `cue_classifier.py` (rules-only)
☐ `agents/observer.py`, `scribe.py` (+sparse separation), `retriever.py`, `curator.py`
☐ Bounded queue + workers, CORS for the UI origin, `/health` shape, `TOTTIE_API_TOKEN` auth on every route

25.5 Phase 4 — MCP

☐ `mcp/server.py`: 9 tools (incl. `memory.confirm`) + 2 resources + `tottie_context_card`
☐ `--list-tools` prints the surface

25.6 Phase 5 — Round-Trip

☐ `POST /write` → gated store → `POST /recall` → Card + confidence
☐ `GET /events` SSE stream for the Panel

☐ `scripts/seed_demo.py`: offline demo seeding (never imported by runtime code; auto-seeding is banned)

25.7 Phase 6 — UI

☐ `TottieFace` (5-state machine + frame-set contract + placeholders)
☐ `ChatStream`, `MemoryPanel`, `RoomMap`, `AblationToggle`, why-surfaced line, Sleep button
☐ Placeholder sprite frames in `ui/public/tottie/<state>/` so the app runs before final art lands
☐ BrainGraph (3D force graph: live nodes, firing path, story line; 2D fallback ready; web3d gates passed)

25.8 Phase 7 — Reconcile / Tags / Identity / Links / Rank

☐ Reconsolidation (`valid_to` / `supersedes`); labile-on-access
☐ Hierarchical tags / rooms; identity allowlist
☐ `memory_links` written on tag/entity/supersede events
☐ Ranker + Compressor: utility decay + ranking (never delete); access boosts utility (testing effect)

25.9 Phase 8 — Cognition + Integration + Eval

☐ Activator (spreading activation, priming, decay) + Completer (rule-based cluster grouping)
☐ Metacognitive Evaluator (confidence) + Prospective Queue (`memory.intend`)
☐ Consolidator (on-demand Sleep: replay $\rightarrow$ semanticise $\rightarrow$ rank/compress)
☐ Mount MCP in VS Code / Cursor (three bodies)
☐ `eval/harness.py` + `scenarios.jsonl` (ON/OFF ablation + consolidation check)
☐ Special checks: revival, surprise-gating, consolidation, quarantine (see *Roadmap*)

25.10 Phase 9 — Film

☐ 3-minute screen-capture video (captions + music), incl. the Sleep beat
☐ 5-minute hallway usability test before filming (a friend drives the UI thinking out loud)
☐ README final; recompile docs; update the compliance ledger
☐ Sync every deck number to measured harness actuals (no aspirational figures on camera)

25.11 Phase 10 — Submit

☐ Make repo public; video link accessible
☐ Claim Meta credits ($<$ Wed Nov 4, 9 PM ET)
☐ Submit via the CV platform

Chapter 26

Demo Script (3 minutes, screen capture + captions + music)

Format: pure screen capture, on-screen captions, a music bed. No talking head. Total runtime **3:00**. Every caption is written to be readable on a phone.

26.1 Shot List

Time	Beat	On screen
0:00	Cold open	Black. Caption: *"A year of conversations."* A plain agent: *"Hi — I don't know you."* Music swells.
0:12	The problem	Restart = amnesia; the agent re-asks your name. Caption: *"Long context isn't memory."*
0:30	The reveal	Sprite wakes; Panel pins Identity `birthday = Nov 7`. Caption: *"It just…knows."*
0:52	Write-time magic	*"I'm vegetarian now"* → Panel: `SUPERSEDE` (old struck through); a new fact → Observer verdict + Scribe `NEW` live.
1:14	Why this surfaced	Command: *"implement feature A"*. Panel: `cue=COMMAND` → `seed #project/A` → `spread 1 hop`. Caption: *"It doesn't search. It associates."*
1:32	**The constraint we refused**	**A memory drops out of the active set: "utility 0.06 → archived" (greyed, but present). Then the cue arrives — it returns instantly. Caption: *"We didn't copy the forgetting. We copied the finding."* Then: *"Not a better memory — a memory without the bug."***
1:52	Three bodies	Cut to VS Code: the coding agent already knows the stack; it hits a bug; **autosave** a lesson; a new session recalls the fix.
2:08	Sleep	Press **Sleep**. Caption: *"12 moments become 3 lessons."* Panel: *12 episodic → 3 semantic; 4 ranked out*.
2:20	Restart + ablation	Kill the session, reopen, ask an old question → knows. Flip **Tottie OFF** → *"I don't know."* ON → knows.
2:42	Joke + pitch	Caption: *"…everything I was mad at my girlfriend for — and the one thing that matters: her birthday."* → the aspirational line + repo link.

26.2 Narration Captions (verbatim)

1. “A year of conversations.”
2. “It still doesn’t know your name.”
3. “Long context isn’t memory — it’s a tax.”
4. “Tottie applies intelligence at write time.”
5. “It decides what matters before you ever need it.”
6. “Ask it to build something. It doesn’t search — it associates.”
7. “Human memory forgets because it is bounded. Ours isn’t.” (the constraint scene)
8. “We didn’t copy the forgetting. We copied the finding.”
9. “Not a better memory — a memory without the bug.”
10. “One brain. Three bodies. Your chat, your code, the panel.”
11. “Then it sleeps — and generalises.”
12. “Turn it off, and it forgets. Turn it on, and it knows.”
13. “So — I used it to remember everything I was mad at my girlfriend for. And the one thing that actually matters: her birthday.”

26.3 Music and Pacing

Cinematic and rising under the reveal; a small pulse on the “spreading activation” beat; a *brief quiet* under the constraint scene (let the words land), then a lift on “memory without the bug”; a calm passage under Sleep; a beat drop on the ablation flip; a warm settle under the joke. Cuts land on caption boundaries. Keep on-screen text large; assume a phone screen.

26.4 Filming Protocol (known vs unknown scenarios)

Known (scripted): every scripted take gets a **shot card** written *before* rolling — fixed seed data, fixed turns, and the expected Panel + graph states (which badge, which nodes, which path lights). Temperature 0, pinned model ids, same seed every take. **Unknown (exploratory):** unscripted conversation, verified *post-hoc* against the session log (did it form memories? does the recap hold? does the story path check out?); run several, keep the best. **Every take** logs take number + seed + model ids + verdict in a run sheet; takes are reviewed the same evening, R-gate style, and only reviewed takes enter the edit.

Chapter 27

Plan Audit (Numbered Outline and Method)

This chapter makes the plan *auditable*. Every part of it carries a stable ID so a second reviewer (human or agent) can judge it section by section without asking for context.

27.1 Audit Method

For each numbered item we record one verdict against six fixed axes:

Axis	Question
Feasibility	Can one person build this Oct 27–Nov 4 alongside work and study?
Dependency	What must exist first? Does it already exist (Reuse Map)?
Risk	What breaks it — live on stage, or under judge scrutiny?
Rubric	Which judging column it serves (Impact / Meta tooling / Technical depth / Demo).
Cost	Effort: S (hours) · M (a day) · L (multi-day).
Verdict	Keep · Improve · Cut · Add · Defer.

27.2 Independent-Review Questions (for third-party judges)

1. Does it satisfy the problem statement (*Build for Your World* / *Push the Frontier*)?
2. Is Meta's tooling central, not decorative?
3. Is the engineering sound and thought through?
4. Is the demo compelling *and live-safe*?
5. Is it clearly new work, distinguishable from reuse?
6. Is anything here a banned anti-project?
7. Are the claims measurable (not vibes)?
8. Is the scope honest for 9 solo days?

27.3 The Numbered Outline

1. **Meta & scope** — event/dates; repo/branch/commit rule; decision log; scope note.
2. **Strategy** — thesis; positioning vs mem0/Zep/Letta; differentiators; problem statement.
3. **Demo design** — three bodies; the money shot; video style; **the Sleep beat**.
4. **Architecture & cognition** — the ten components; tiers; write path; reconcile; tier routing; recall path; Context Card; probe/tags; concurrency; **cue classifier**; **activator**; **completer**; **consolidator**; **salience gate**; **reconsolidation**; **neuroscience basis**.

5. **Data & storage** — `memories`; `memory_events`; indexes; pgvector decision; embeddings; init; owner scoping; **`memory_links`**; **principle/arousal/access fields**.
6. **Intelligence layer** — Observer; Scribe; Curator; providers; model choice/cost; sampling; **consolidation prompts**.
7. **Interfaces** — HTTP surface (`/consolidate`); MCP tools (`memory.sleep`); resources; prompt; SSE events; error/abstention.
8. **UI/UX** — layout; panel blocks; sprite state machine; ablation; room map; typography; **why-surfaced line + Sleep button**.
9. **Evaluation** — scenario set; ablation method; metrics; harness; rubric mapping; **consolidation check**.
10. **Execution plan** — MUST/SHOULD/STRETCH; phase schedule; gating; mandates.
11. **Media** — shot list; deck; music/captions; viral plan.
12. **Repo & tooling** — tree; reuse map; orch/env; docs structure; build commands; gitignore/Makefile; **cognition modules**.
13. **Compliance** — rules mapping; contribution ledger; prize targeting; banned-avoidance.
14. **Risk** — register; live-demo mitigations; **cognition scope-creep mitigations**.
15. **Verification** — test plan; dry-run checklist; orch checks.
16. **Open items** — sprite frames; Meta keys; sampling; name/branch.
17. **Appendices** — rules extract; glossary; sources.

27.4 Audit Ledger

The per-item verdicts are maintained alongside this specification. Items are resolved to `Keep`/`Improve` before the spec is frozen for build. The cognition-model IDs added in v3 (**4.9–4.15, 5.7–5.8, 6.6, 8.6, 9.5, 3.6**) are recorded as `Add`, with the two open items resolved: the Consolidator is SHOULD/on-demand, and the Sleep beat enters the video.

27.5 v6 Audit Ledger (pre-build, 2026-10-26)

Findings from the front-to-back audit; each fix is already folded into this spec.

ID	Finding	Fix (where)
A1	No HTTP schema: endpoints named, shapes nowhere	new *HTTP API* chapter
A2	Driver mismatch (docs: asyncpg, requirements: psycopg)	standardised on psycopg `AsyncConnectionPool`
A3	Idempotency unimplementable: `hash` had no unique index	unique `(owner, hash)` index in schema
A4	`TOTTIE_API_TOKEN` specified but in no env file	added to `.env.example` + all orch profiles
A5	No CORS plan, but UI calls core across origins	CORS section in *HTTP API*; wired in P3
A6	Quarantine covered identity/convention only	widened to decision/lesson; episodic stays labelled
A7	No way to confirm a quarantined memory	`/memories/{id}/confirm` + `memory.confirm` (9th tool)
A8	Over-fetch was the only filtered-search defence	iterative scan + `ef_search=100` primary (pgvector $\geq$ 0.8)
A9	No HNSW params, tokenizer, rate values, tuning defaults	all pinned (see *Performance*, `.env.example`)

A10	2 write calls budgeted; LLM reachable on read	one shared call; rules-only classifier; 0 on read
A11	Stale “prune” wording in 3 chapters vs D4	rewritten to ranked-out
A12	Deck never mentions Meta (rubric risk)	Muse Spark bullet; ordered open-weights fallback
A13	Prospective firing needed an unplanned scheduler	poll-on-recall/write, no scheduler
A14	Transactive/Completer/RIF/imagination undecided	cut / rule-based / flag-off / deferred
A15	Eval: 10 scenarios, harness + checks unspecified	20 scenarios + harness contract in *Roadmap*
A16	No compliant demo seed path (auto-seeding banned)	offline `seed_demo.py` specified (P5)
A17	No local Postgres/pgvector; Python 3.9; no Ollama	P3 setup: brew python, postgres, pgvector

27.6 v7 Research Ledger (3D brain graph + retrieval story, 2026-10-27)

Notebook grew to 44 sources (context-dependent memory, free recall, source-monitoring error, mem0 paper); two answered queries designed the story feature. Library verdict: `react-force-graph-3d` (props verified against the README), 2D sibling as film-day fallback, literal brain mesh cut as scope creep.

ID	Finding	Fix (where)
B1	Memory formation invisible on camera	Brain Graph spec in *Tottie UI* (live nodes, firing path)
B2	No graph data API; recall has no path ids	`GET /graph` + `path:[ids]` in *HTTP API*
B3	“Why did this pop?” unanswered	3-stage story chain in *Recall Path* (overload $\rightarrow$ context $\rightarrow$ margin + unroll)
B4	No filming discipline for takes	Filming Protocol in *Demo Script* (shot cards, take log, same-evening review)
B5	W4 missing the graph build	wave row + validation updated in *Execution Plan*; web3d gates required

27.7 v8 Planning Ledger (Joel + Shape Up + WBS + risk audits, 2026-10-27)

Four notebook queries audited the execution plan against the new planning literature (50 sources). Real gaps filled; over-simplified advice overruled on record.

ID	Finding	Fix (where)
C1	No one-step build; `verify` swallowed failures (`|| true`)	Makefile: docs+deck+UI+pytest, fails loud
C2	No CI	`.github/workflows/ci.yml` (pytest + pgvector service, UI build)
C3	No bug database	GitHub Issues (repo stays LaTeX-pure) + per-gate bug lists

C4	No hallway usability test	5-minute test in P9, before filming
C5	35 h single-point, no buffer, no re-estimation	27 waves + 4 gates + 4 buffer; re-estimate at gates; hour-25 breaker
C6	Risks implicit, untriggered	R1–R6 register with triggers in *Execution Plan*
C7	Eval wave unshaped (grab-bag)	pre-shaped exit: harness + 4 checks + 40 (min 22) scenarios

27.8 v9 Final Ledger (pre-development consistency sweep, 2026-10-27)

Full cross-read of the spec the night before code. (F-series: D is taken by locked decisions.)

ID	Finding	Fix (where)
F1	Suite count stale (“ships 20”, is 22)	corrected in *Roadmap*
F2	Transactive CUT in roadmap, STRETCH in two roster tables + prose	marked CUT in *Architecture*, *Cognition*
F3	Graph libs missing from UI deps; three-duplication + TS-shim risks	pinned in `package.json` (verified ranges); shim specced
F4	`make mcp` used system python, no core path	venv python + PYTHONPATH
F5	MCP auth was “validates its caller” (no mechanism)	fail-loud token + local-process boundary in *Security*, *MCP*
F6	SSE auth impossible (EventSource has no headers)	`?token=` on `/events` in *HTTP API*
F7	`turn_id` vs `event_id` conflated	server assigns `event_id`; turn echoed
F8	Entity extraction unspecified	rules-only: tags, @mentions, quotes (*Cue Retrieval*)
F9	Budget math inconsistent (28+7 vs wave sum)	27 waves + 4 gates + 4 buffer = 35, per-wave hours stated
F10	Deck numbers could go stale vs harness	R3 + P9 sync deck to measured actuals
F11	CI lacked a postgres client for `init_db.sh`	apt-install step in workflow
F12	Brain graph placement unspecified	Panel tabs Timeline/Graph in *Tottie UI*

27.9 v10 Incident Ledger (the YOLO false-block, 2026-10-27)

A real failure from this build: the agent insisted for minutes that escalation was blocked while YOLO mode was on the entire time. Dogfooded as Gallery row 11 and scenario s23.

ID	Finding	Fix (where)
G1	Agent repeated a false self-model across turns; nothing persists the correction	Gallery row 11 in *Why* (dogfood story for the demo)
G2	No regression test for lesson persistence	s23 procedural-lesson scenario
G3	Honest scope: memory makes it a once-ever failure, not a never-error	recorded here; inference errors in-run are out of scope

27.10 v11 Build Ledger (Meta live + keys cascade + W0 green, 2026-10-27)

ID	Finding	Fix (where)
H1	Meta endpoint unproven; key unverified	live call OK (5.3 s); base/model in env; reasoning notes in *Providers*
H2	“I lost my keys” needs recall but parses as STATEMENT	loss/need patterns take QUESTION policy (*Cue Retrieval*) + s24
H3	Vague-cue cascade unmapped to components	reinstate $\rightarrow$ compound probe, converge $\rightarrow$ Activator, complete $\rightarrow$ Completer (this row)
H4	W0 built: 7/7 pytest, TCP boot OK, 2 missing deps found	code in `core/`; P0–P2 boxes checked in *Todo*